Task 5 : Working with SLURM - Scripting & Kubeflow

Date: 27-03-2026

PART 1 — SLURM

Procedure

The purpose of this study was to analyze how job scheduling is performed using the SLURM workload manager in a container environment. To perform the experiment, the SLURM cluster was created based on the Docker platform that comprised a controller node as well as compute nodes. In order to test SLURM functionality, a batch job consisting of matrix addition implemented with Python was run.

Step-by-Step Explanation

The SLURM cluster setup was done using Docker-based cluster setup.

```
git clone https://github.com/giovtorres/slurm-docker-cluster.git
cd slurm-docker-cluster
docker compose up -d
```

These clusters running were checked by doing the following commands:

```
docker ps
docker exec -it slurmctld bash
sinfo
```

A simple Python script to add two matrices:

```
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

result = [[A[i][j] + B[i][j] for j in range(2)] for i in range(2)]
print(result)
```

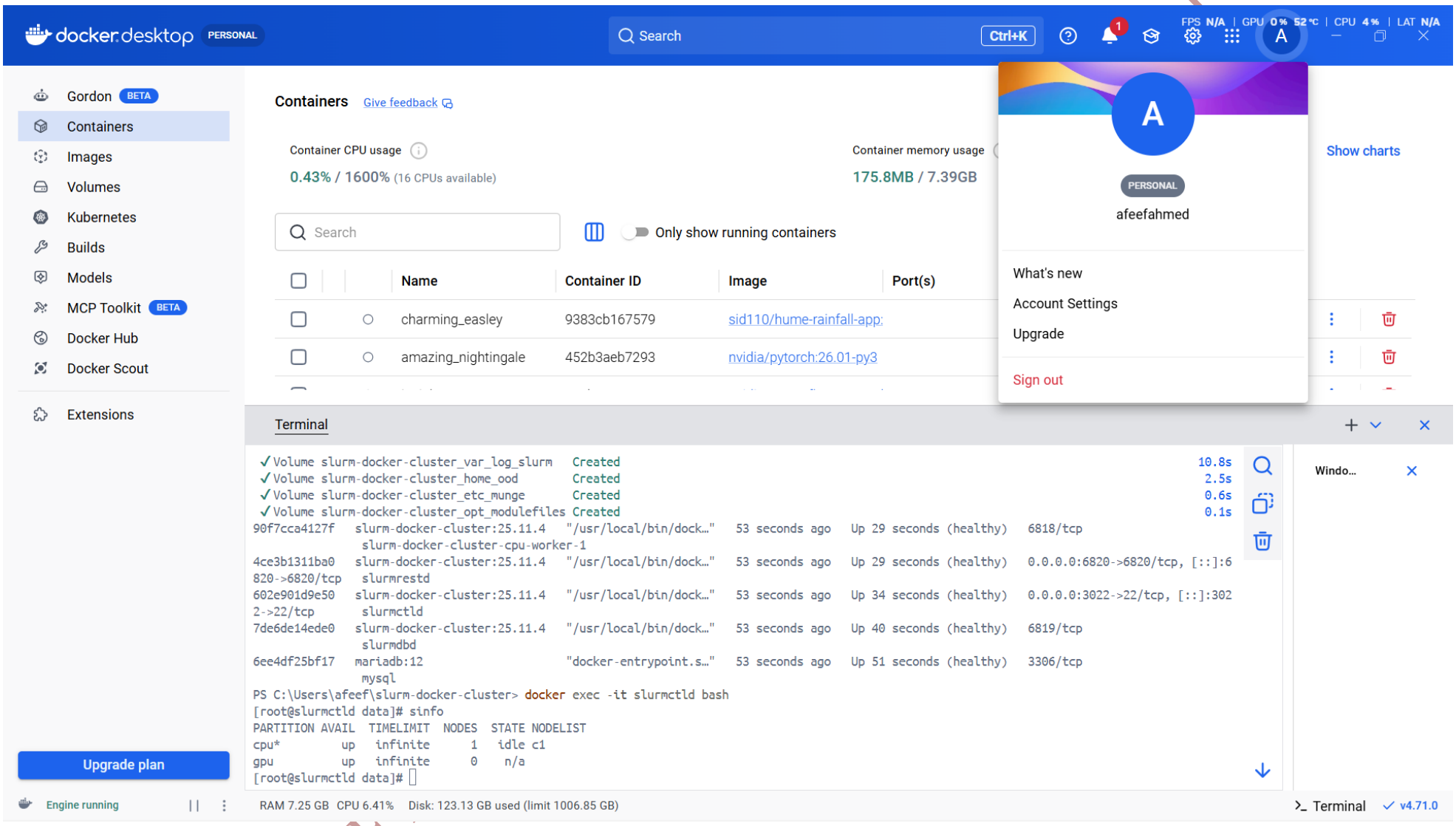

Fig 33: Docker Containers Running (SLURM Cluster)

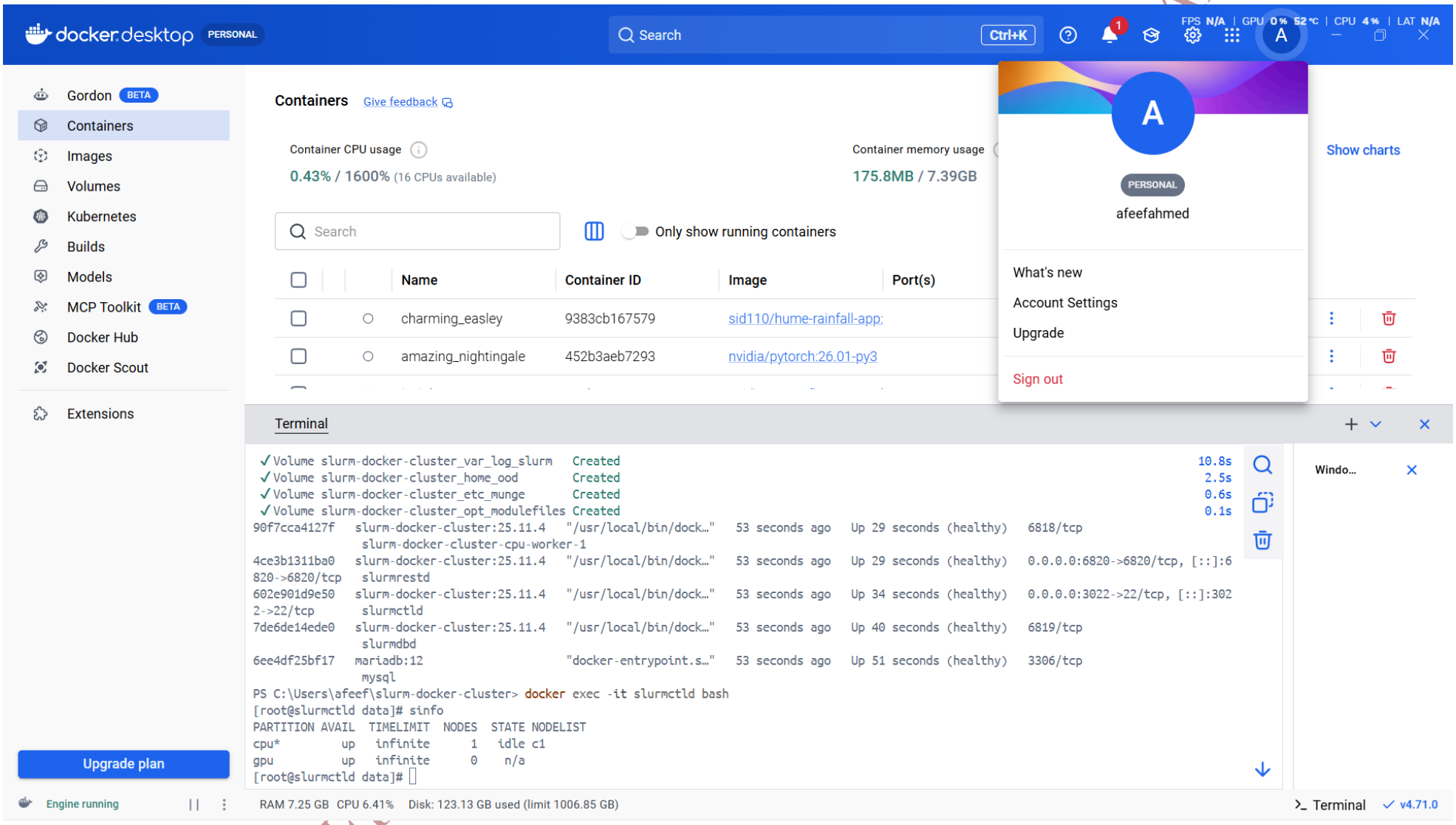

Fig 34: SLURM Cluster Status (sinfo)

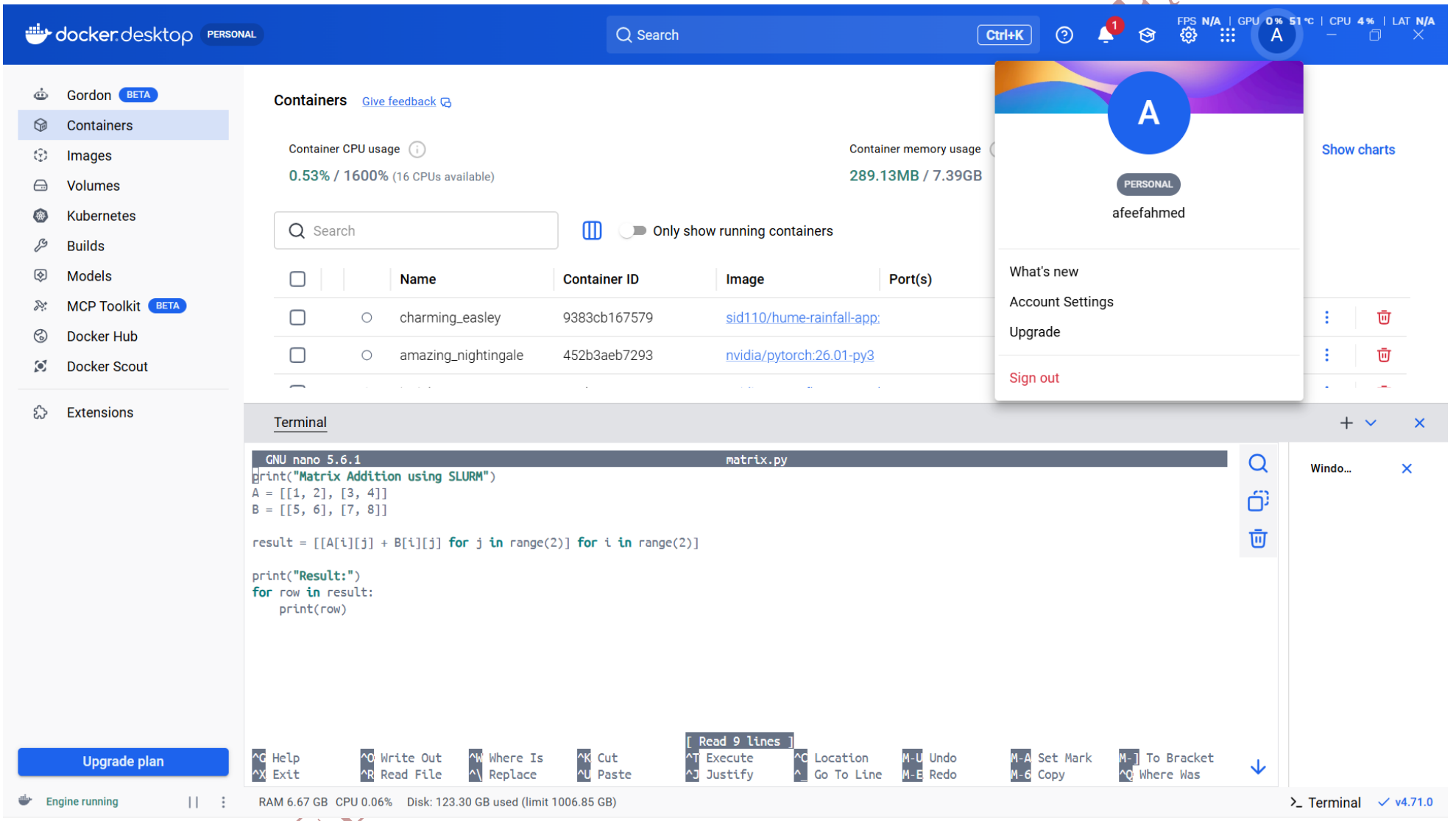

Fig 35: Matrix Program in SLURM

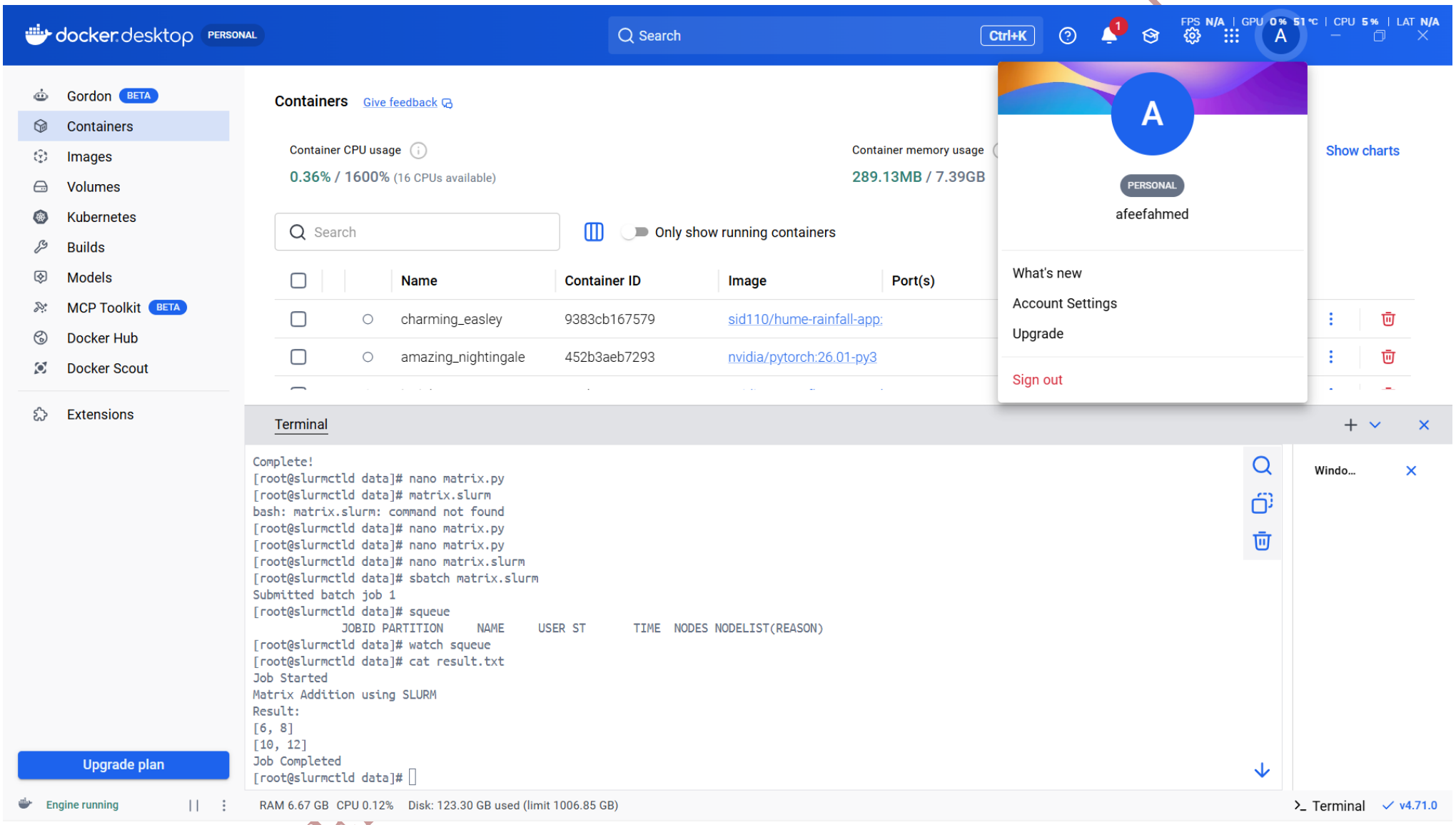

Fig 36: SLURM Job Output

A batch script with SLURM commands was developed as follows:

```
#!/bin/bash
#SBATCH --job-name=matrix_add
#SBATCH --output=result.txt
python3 matrix.py
```

The script was successfully run:

```
sbatch matrix.slurm
cat result.txt
```

As seen from the result, the experiment succeeded as SLURM was able to schedule and execute the command in question.

PART 2 — Kubeflow

Procedure

The goal of the experiment was workflow automation with Kubeflow Pipelines. Thus, a matrix addition component had to be created and compiled into a YAML file describing the workflow process.

Step-by-Step Explanation

Firstly, we installed Kubeflow Pipelines SDK to allow us to define our workflows:

```
pip install kfp
```

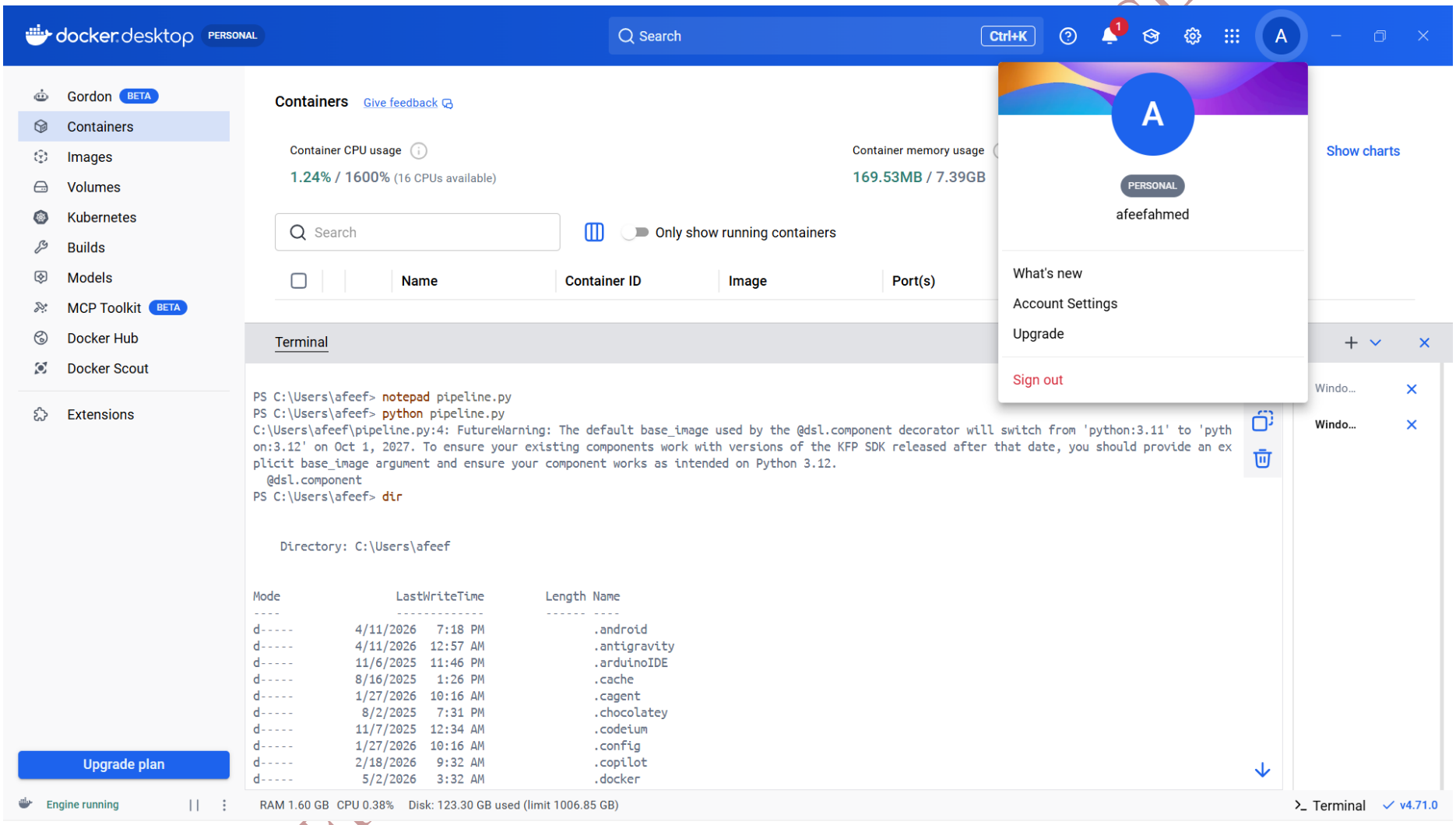

Fig 37:Kubeflow Pipeline Script

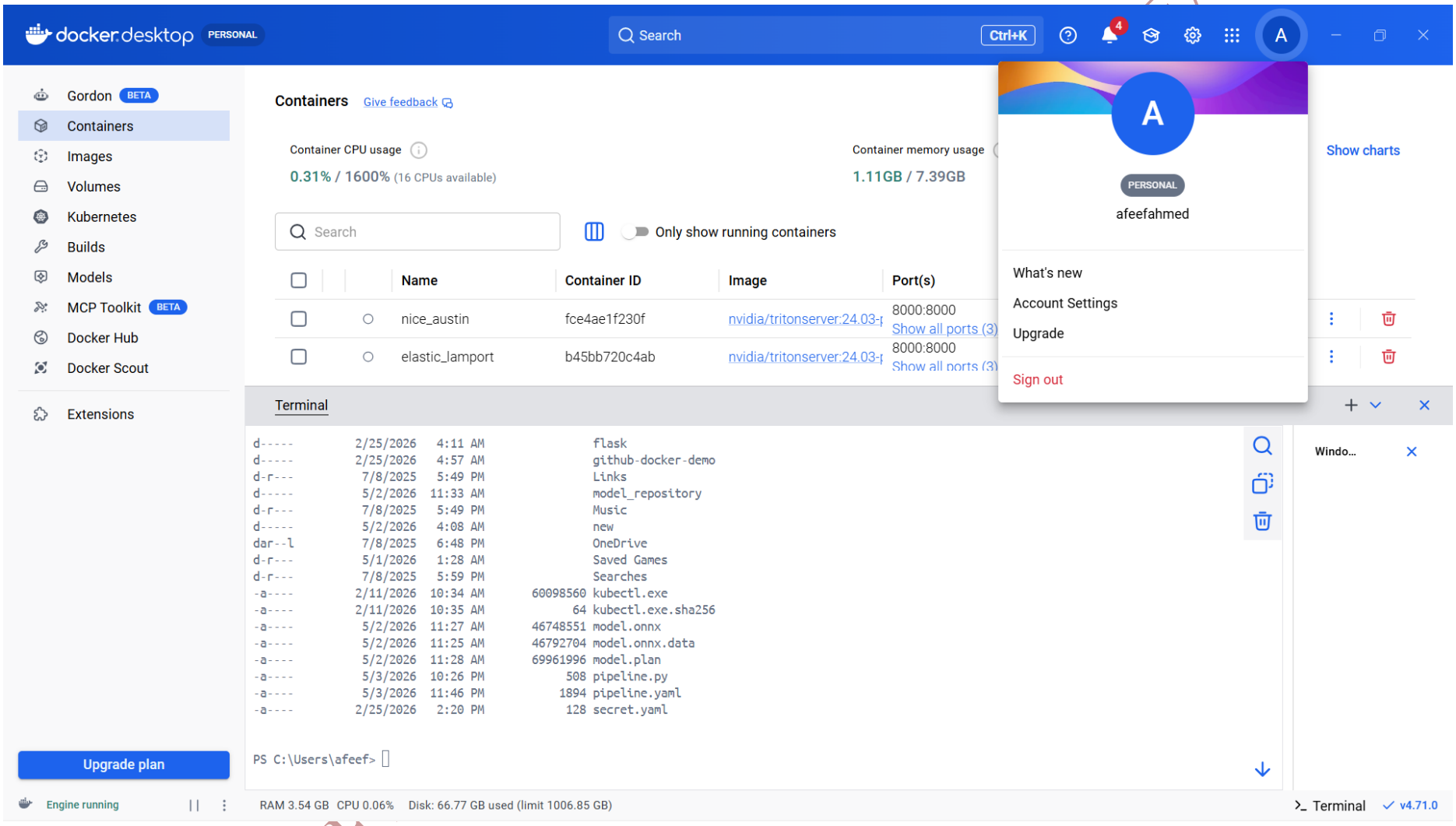

Fig 38:Kubeflow Pipeline YAML Listing

A script in Python was developed to implement a reusable module to perform matrix addition. This involved building logic for matrix addition:

```
from kfp import dsl
from kfp.compiler import Compiler

@dsl.component
def matrix_addition():
    print("Matrix Addition using Kubeflow")

    A = [[1, 2], [3, 4]]
    B = [[5, 6], [7, 8]]

    result = [[A[i][j] + B[i][j] for j in range(2)] for i in range(2)]

    print("Result:")
    for row in result:
        print(row)

@dsl.pipeline(name="matrix-addition-pipeline")
def pipeline():
    matrix_addition()
```

Execution:

```
python pipeline.py
```

The resulting pipeline.yaml indicated that the pipeline was successfully created..

Task 6 : Using Jupyter Notebook and PyTorch for Tensor Operations and Data-Centric Application

Date: 08-04-2026

Experiment Procedure

This experiment was designed to understand tensor operations and create a data-centric application using Jupyter Notebook and PyTorch. The following steps were executed to complete the task.

Step-by-Step Explanation

Installation of required libraries and opening of notebook.
Performance of tensor operations using Python script.
Creation of data-centric application using linear regression model.
Experiment Steps Explanation

In order to perform the experiment, all the required libraries were installed and Jupyter Notebook was opened:

```
pip install notebook torch torchvision
jupyter notebook
```

After opening the notebook, two tensors were created and several operations like addition, subtraction, element-wise multiplication, and matrix multiplication were executed on them.

```
import torch

a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6], [7, 8]])

print("Addition:\n", a + b)
print("Subtraction:\n", a - b)
print("Element-wise Multiplication:\n", a * b)
print("Matrix Multiplication:\n", torch.matmul(a, b))
```

Further, the data-centric application was prepared by creating a dataset containing linearly related variables and defining a linear model using PyTorch.

```
import torch.nn as nn
import torch.optim as optim

x = torch.tensor([[1.0], [2.0], [3.0], [4.0]])
y = torch.tensor([[3.0], [5.0], [7.0], [9.0]])

model = nn.Linear(1, 1)
```

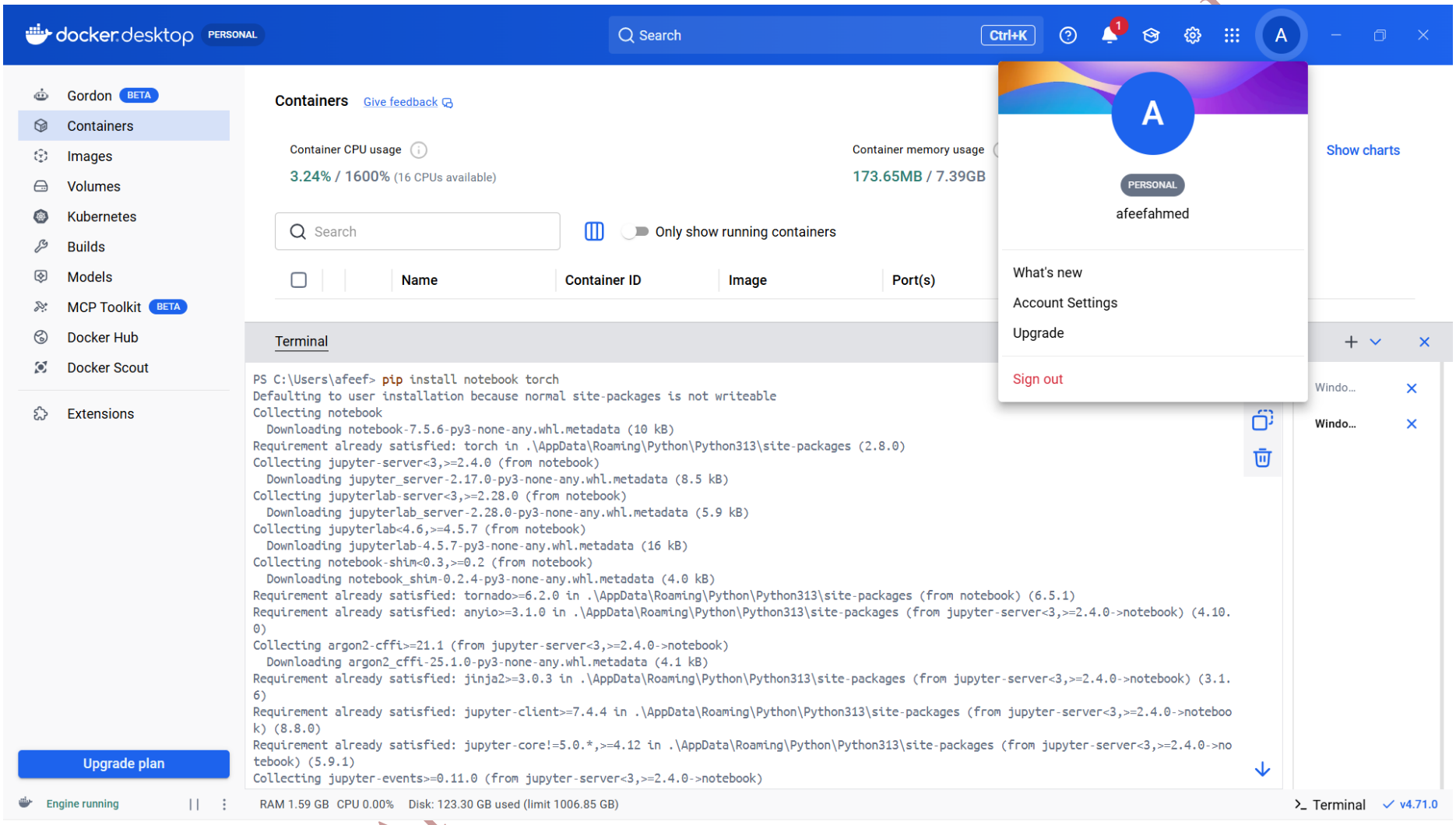

Fig 39:Jupyter Installation Process

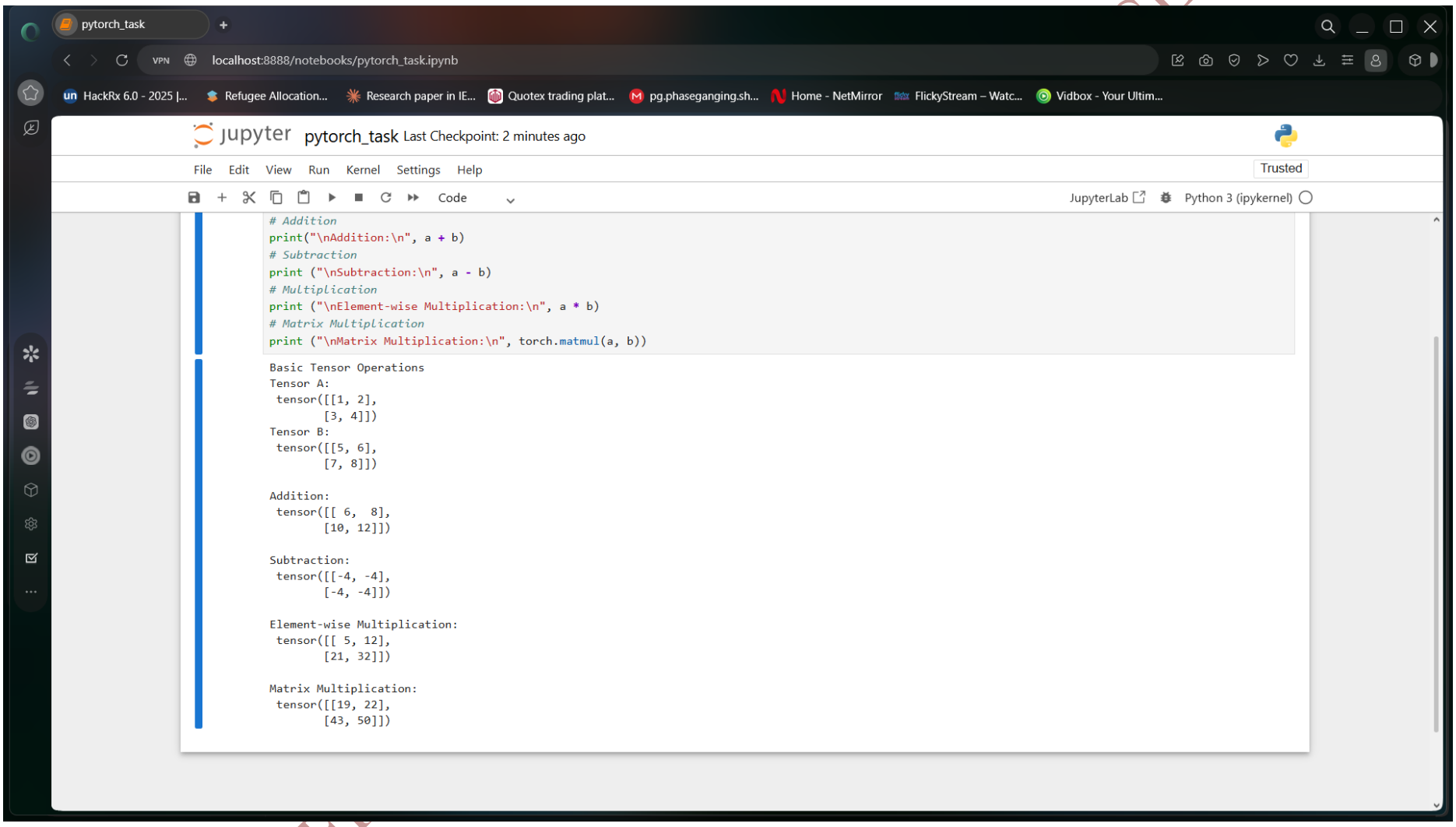

Fig 40:Jupyter Tensor Output

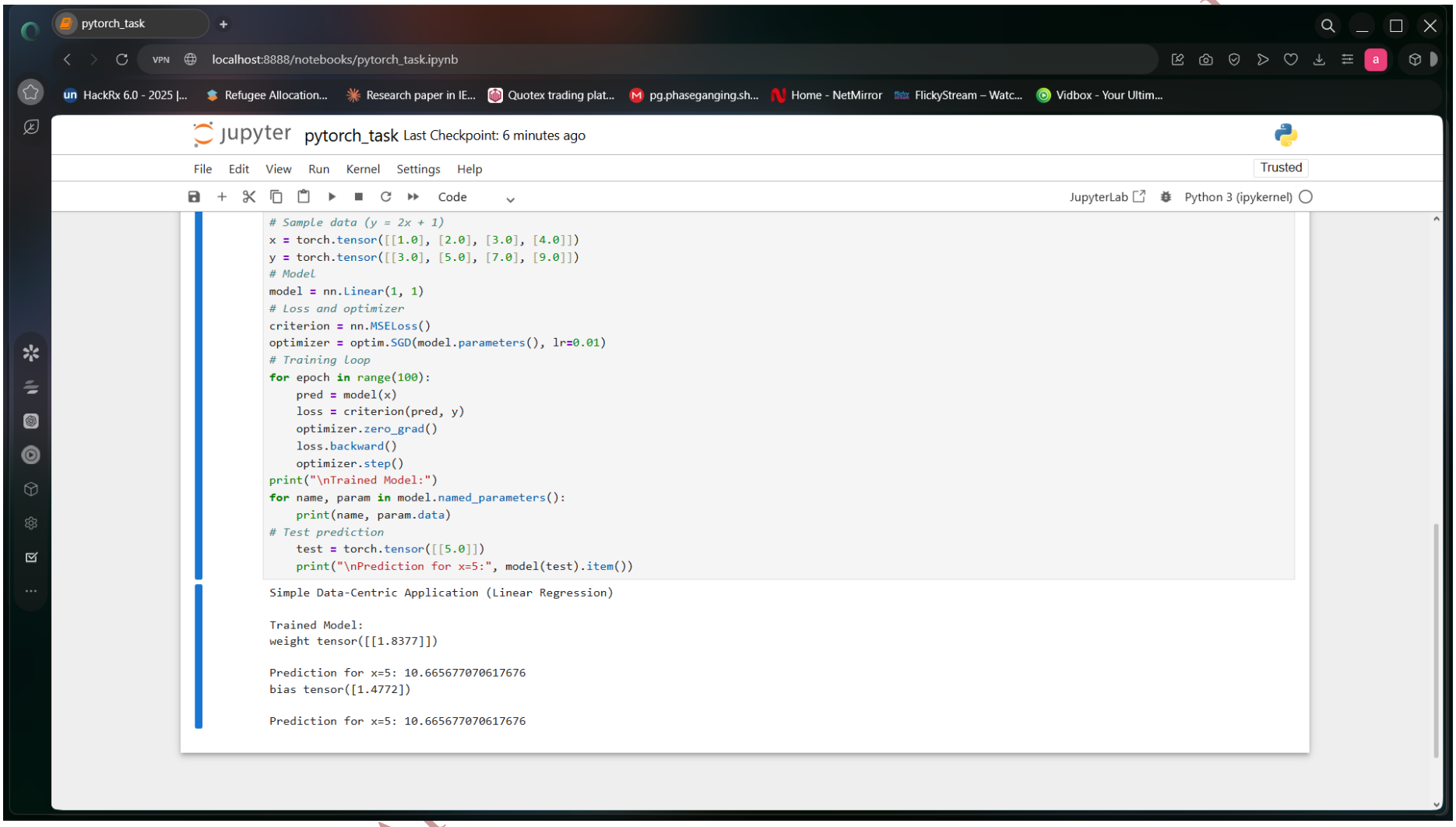

Fig 41:Linear Regression Output

After preparing the dataset and defining the model, the next step involved specifying a loss function and an optimizer in order to train the linear regression model iteratively.

```
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

for epoch in range(100):
    pred = model(x)
    loss = criterion(pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

After training the model using iterative epochs, the final step was to test the trained model using test inputs.

```
test = torch.tensor([[5.0]])
print("Prediction for x=5:", model(test).item())
```

Finally, we checked whether the newly trained linear model gave correct predictions for new inputs.

Task 7 : Testing the GPGPU Computing

Date: 21-04-2026

Procedure

In this experiment, analysis and comparison of CPU-based computation vs GPU-based computation was carried out using PyTorch. The objective was to show the improvements offered by the use of GPUs for parallel computing on large scale tasks.

Step-by-Step Explanation

In order to confirm the availability of a GPU, the following code snippet was executed.

```
import torch

print("GPU Available:", torch.cuda.is_available())
```

Firstly, a large dataset was created. Sorting operation was performed using CPU, and runtime was measured.

```
import time

data = torch.rand(10_000_000)

start = time.time()
torch.sort(data)
cpu_time = time.time() - start

print("CPU Time:", cpu_time)
```

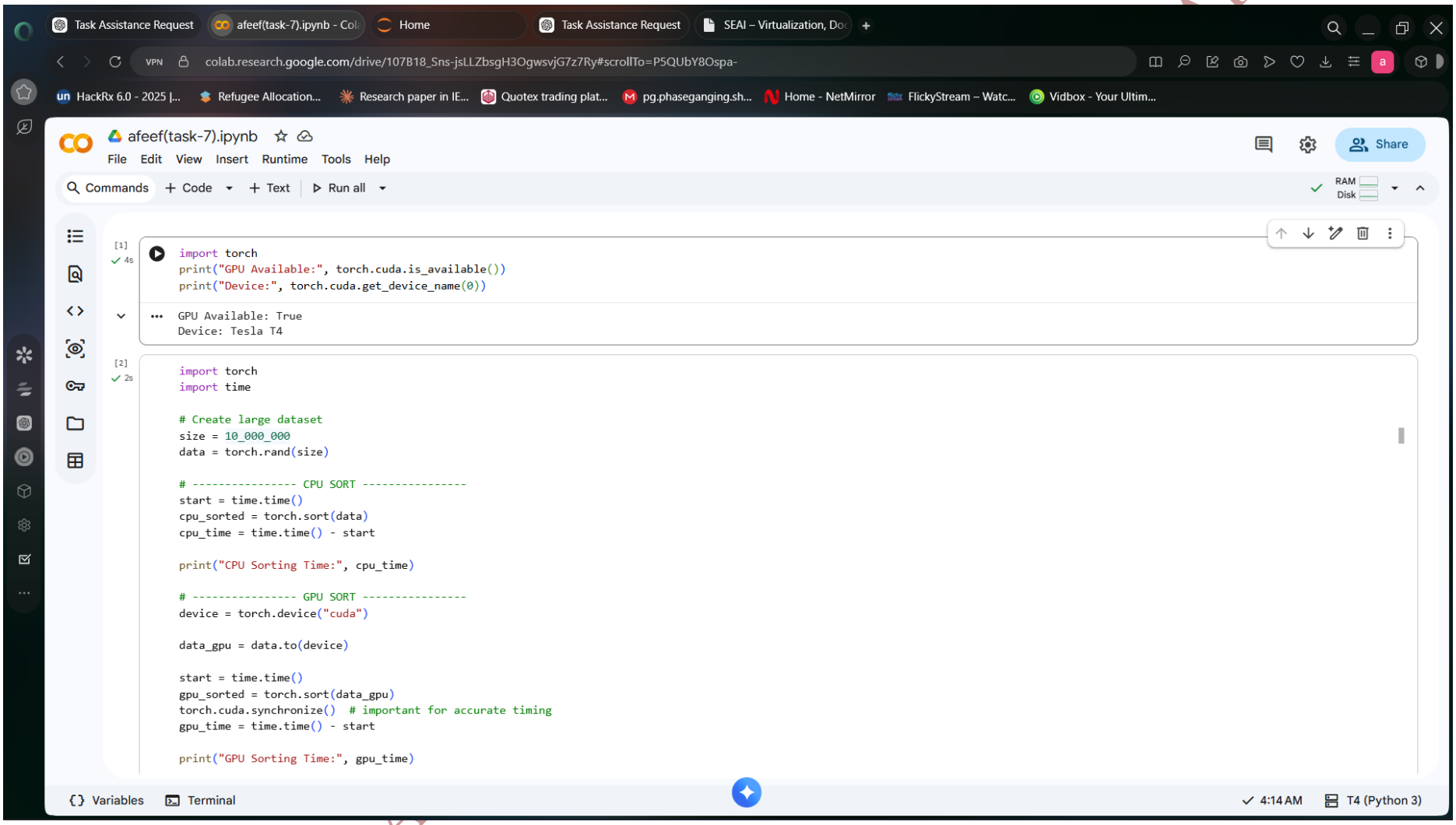

Fig 42:GPU Verification Output

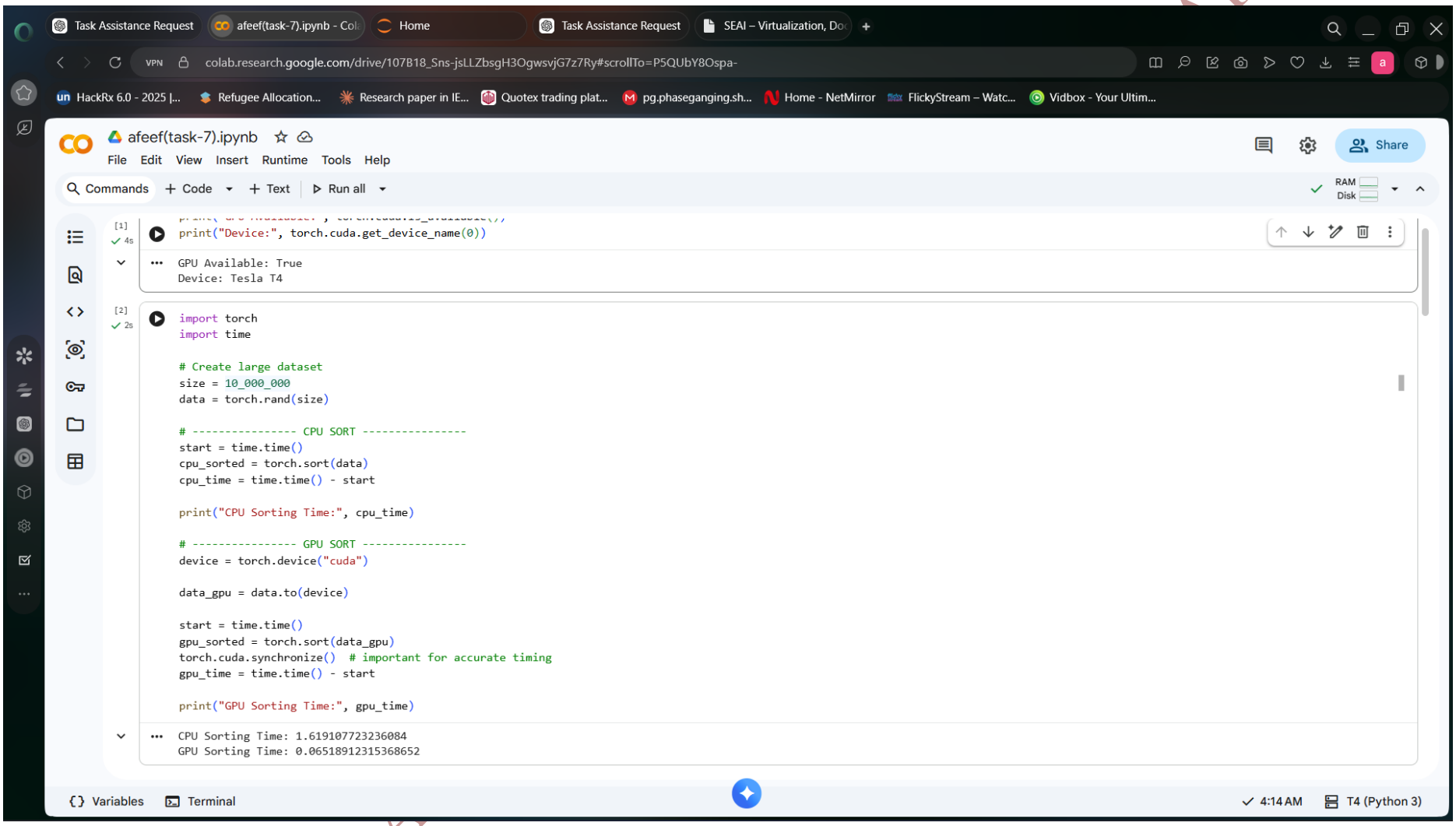

Fig 43:GPU Sorting Output

Then, the same data was moved to the GPU memory, and the sorting operation was again performed to measure runtime.

```
device = torch.device("cuda")

data_gpu = data.to(device)

start = time.time()
torch.sort(data_gpu)
torch.cuda.synchronize()
gpu_time = time.time() - start

print("GPU Time:", gpu_time)
```

As seen from the results, GPU computations were considerably more efficient compared to CPU for large data sets, which demonstrated the benefits of parallel computations.

Task 8 : Testing CUDA-X Tensor Operations, Mixed Precision, and Quantization-Aware Training using PyTorch

Date: 25-04-2026

Procedure

In the current experiment, we explored GPU programming in PyTorch through advanced CUDA tensor operations, mixed precision, and automatic differentiation.

Step-by-Step Explanation

In the following example, the device was selected as either GPU or CPU based on availability.

```
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using:", device)
```

Large tensors were created on GPU and matrix multiplication was applied to them.

```
A = torch.randn(4096, 4096, device=device)
B = torch.randn(4096, 4096, device=device)

C = A @ B
print("Matrix multiplication completed")
```

Autodiff capabilities in PyTorch were tested next.

```
x = torch.tensor(2.0, requires_grad=True)
y = x**3 + 2*x**2 + x

dy_dx = torch.autograd.grad(y, x, create_graph=True)[0]
d2y_dx2 = torch.autograd.grad(dy_dx, x)[0]

print("First derivative:", dy_dx.item())
print("Second derivative:", d2y_dx2.item())
```

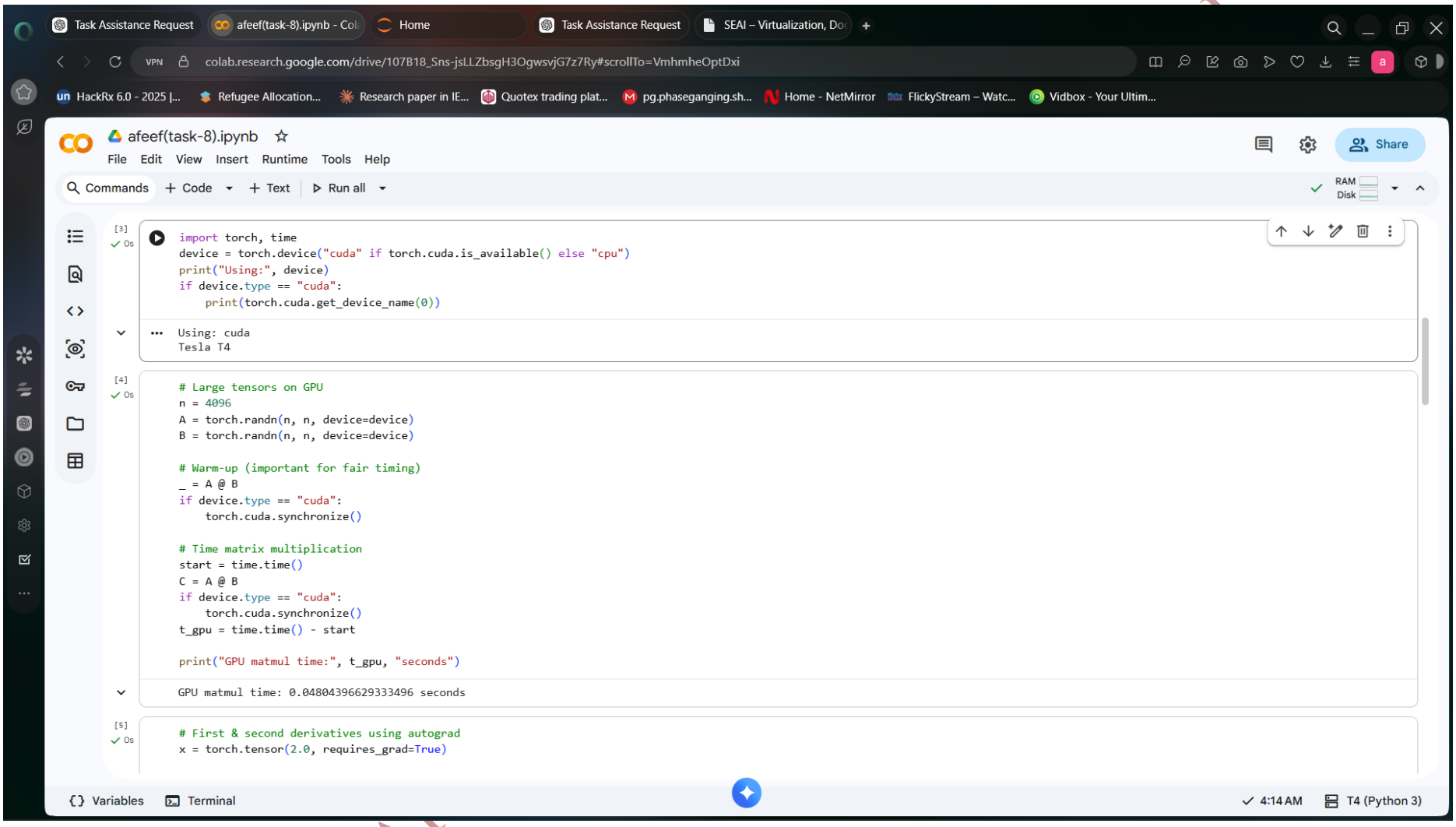

Fig 44:GPU Tensor Operations

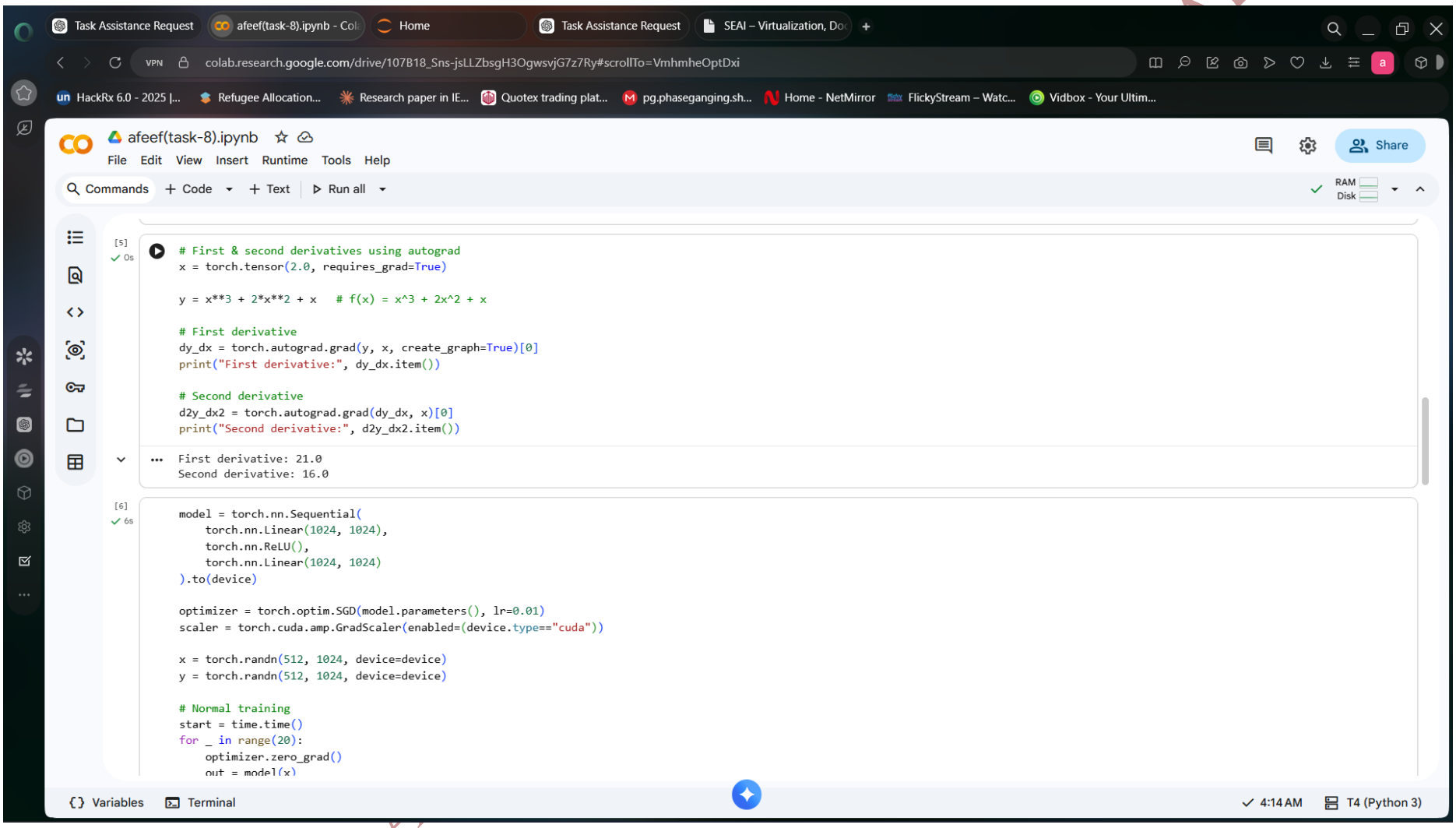

Fig 45:Autograd Derivatives Output

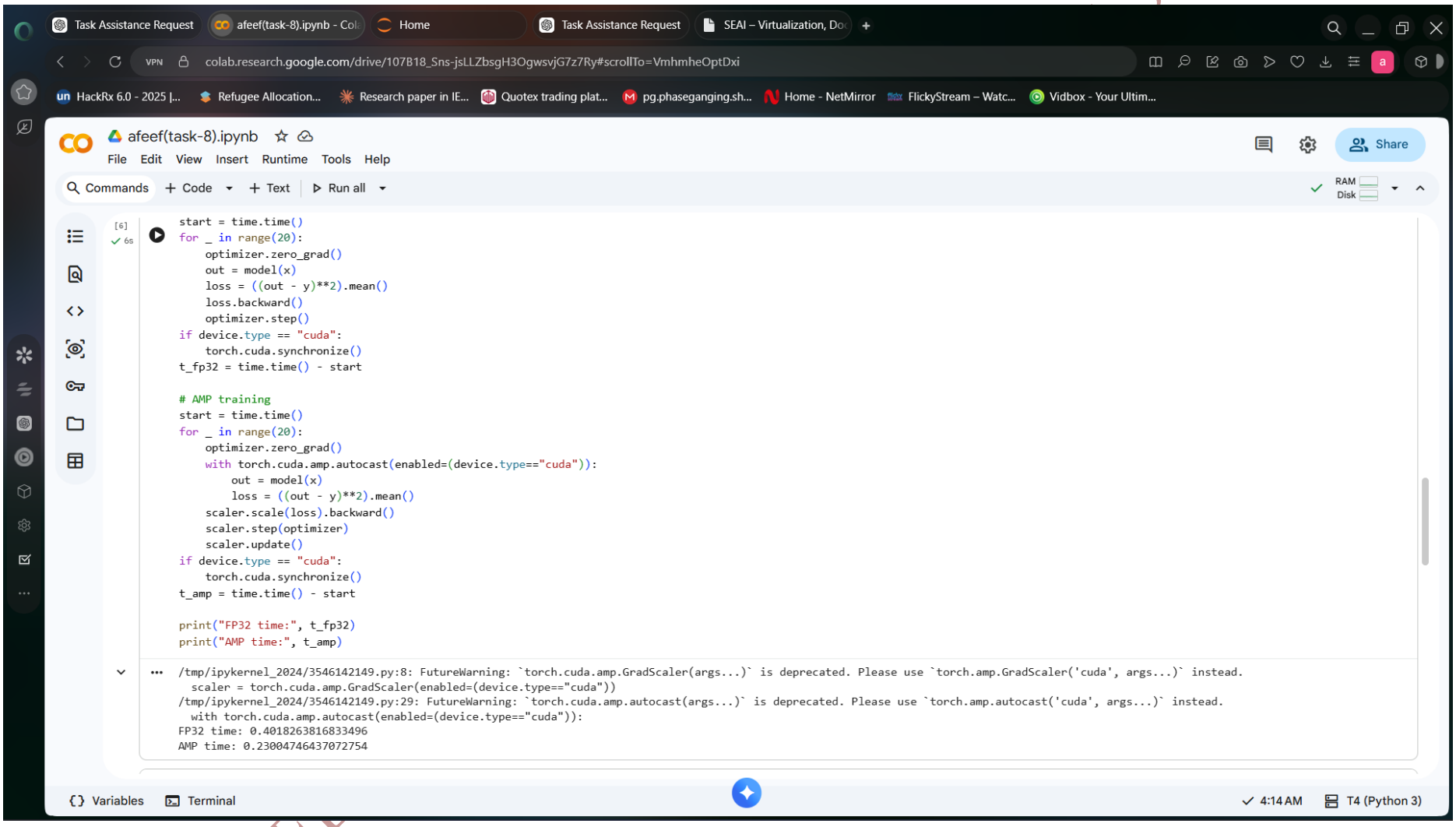

Fig 46:AMP Performance Output

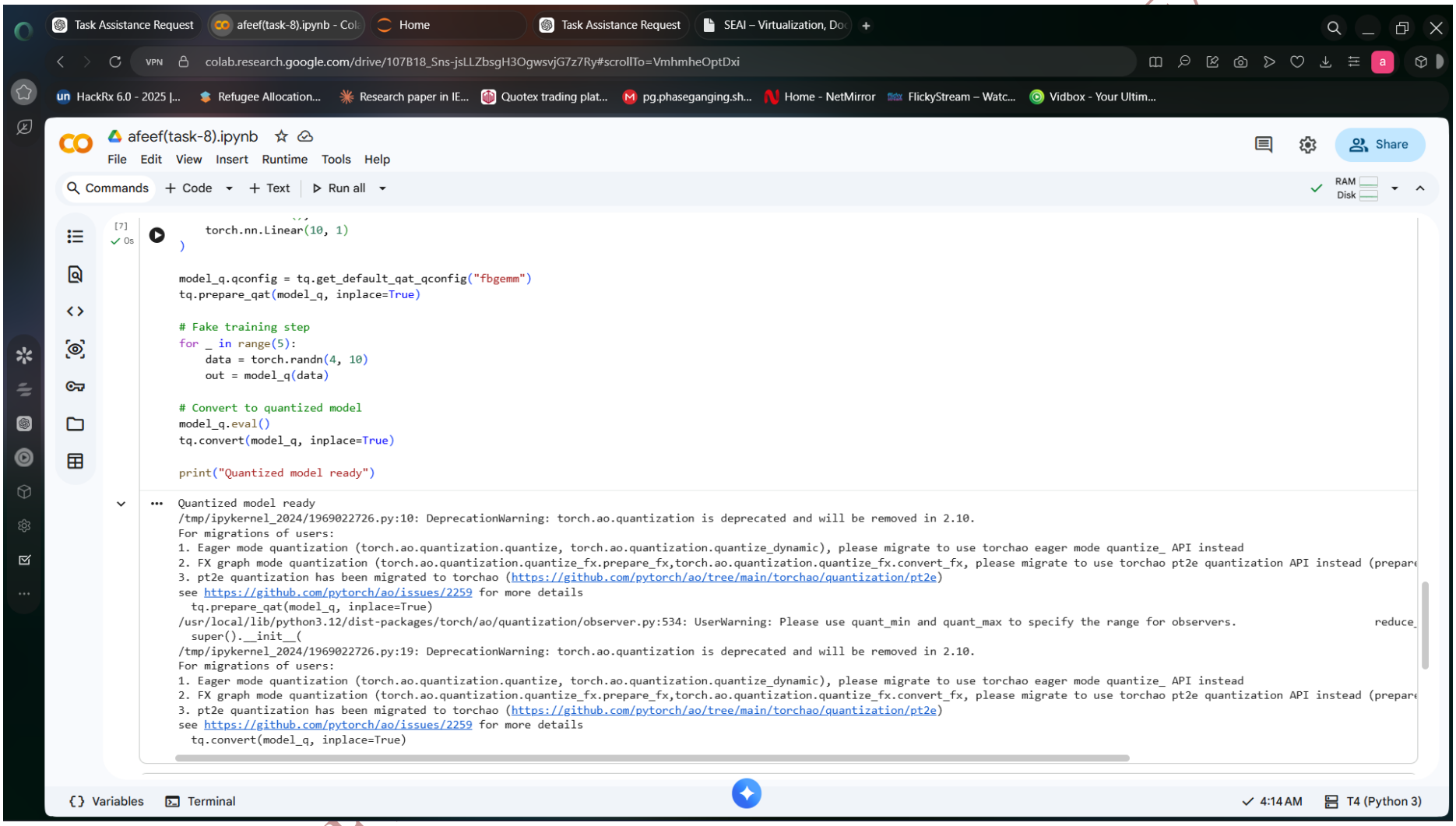

Fig 47:Quantization Output

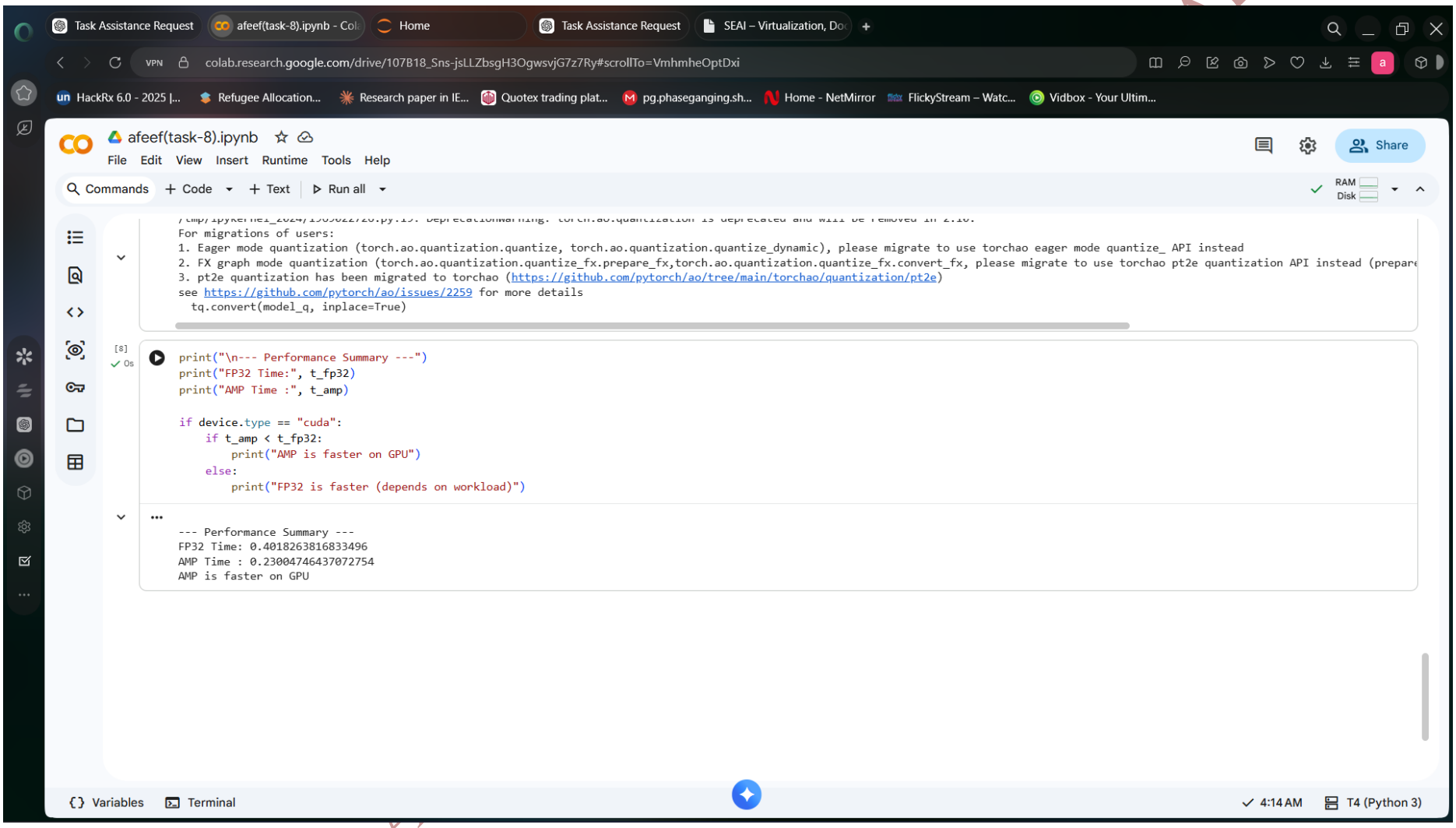

Fig 48:Performance Summary Output

Next, mixed precision using autodiff was tested to increase speed.

```
from torch.cuda.amp import autocast

with autocast():
   result = A @ B

print("AMP computation completed")
```

Finally, quantization-aware training with PyTorch library was tested.

```
import torch.quantization as tq

model = torch.nn.Linear(10, 1)
model.qconfig = tq.get_default_qat_qconfig("fbgemm")

tq.prepare_qat(model, inplace=True)
tq.convert(model, inplace=True)

print("Quantization completed")
```

After this, quantization aware training was demonstrated using PyTorch libraries..

```
import torch.quantization as tq

model_q = torch.nn.Sequential(
    torch.nn.Linear(10, 10),
    torch.nn.ReLU(),
    torch.nn.Linear(10, 1)
)

model_q.qconfig = tq.get_default_qat_qconfig("fbgemm")
tq.prepare_qat(model_q, inplace=True)

for _ in range(5):
    data = torch.randn(4, 10)
    out = model_q(data)

model_q.eval()
tq.convert(model_q, inplace=True)

print("Quantized model ready")
```

There was an improvement in efficiency and reduced computation costs.

Task 9 : Accelerating Neural Network Inferencing: TensorRT & Triton Inference Server

Date: 28-04-2026

Procedure

This experiment was conducted to accelerate neural network inferencing using TensorRT and Triton Inference Server. It involved model conversion, repository creation, inference server deployment, and finally model inferencing.

Step-by-Step Explanation

A model was generated using PyTorch and exported to the ONNX format.

```
import os
os.environ["TORCH_ONNX_USE_DYNAMO"] = "0"

import torch
import torchvision.models as models

model = models.resnet18(pretrained=True)
model.eval()

dummy_input = torch.randn(1,3,224,224)

torch.onnx.export(model, dummy_input, "model.onnx", opset_version=11)
```

This model was then converted to the tensorRT engine.

```
trtexec --onnx=model.onnx --saveEngine=model.plan
```

The next step involved the generation of the model repository and placement of the engine files in the repositor.

```
model_repository/
└── my_model/
    └── 1/
        └── model.plan
```

Configuration for this repository was made where the input/output information were specified.

```
name: "my_model"
platform: "tensorrt_plan"
max_batch_size: 1
```

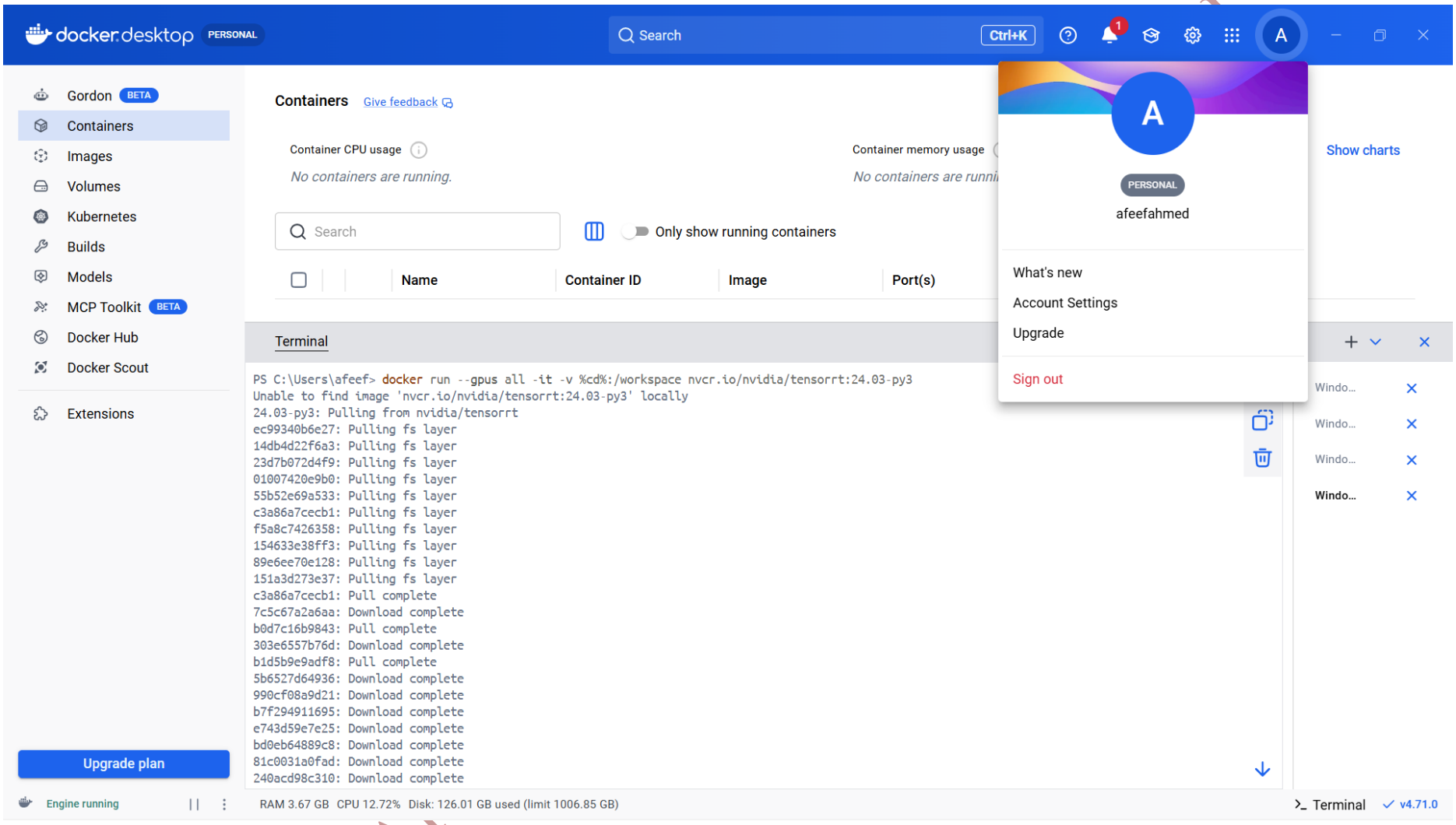

Fig 49:Triton Docker Pull

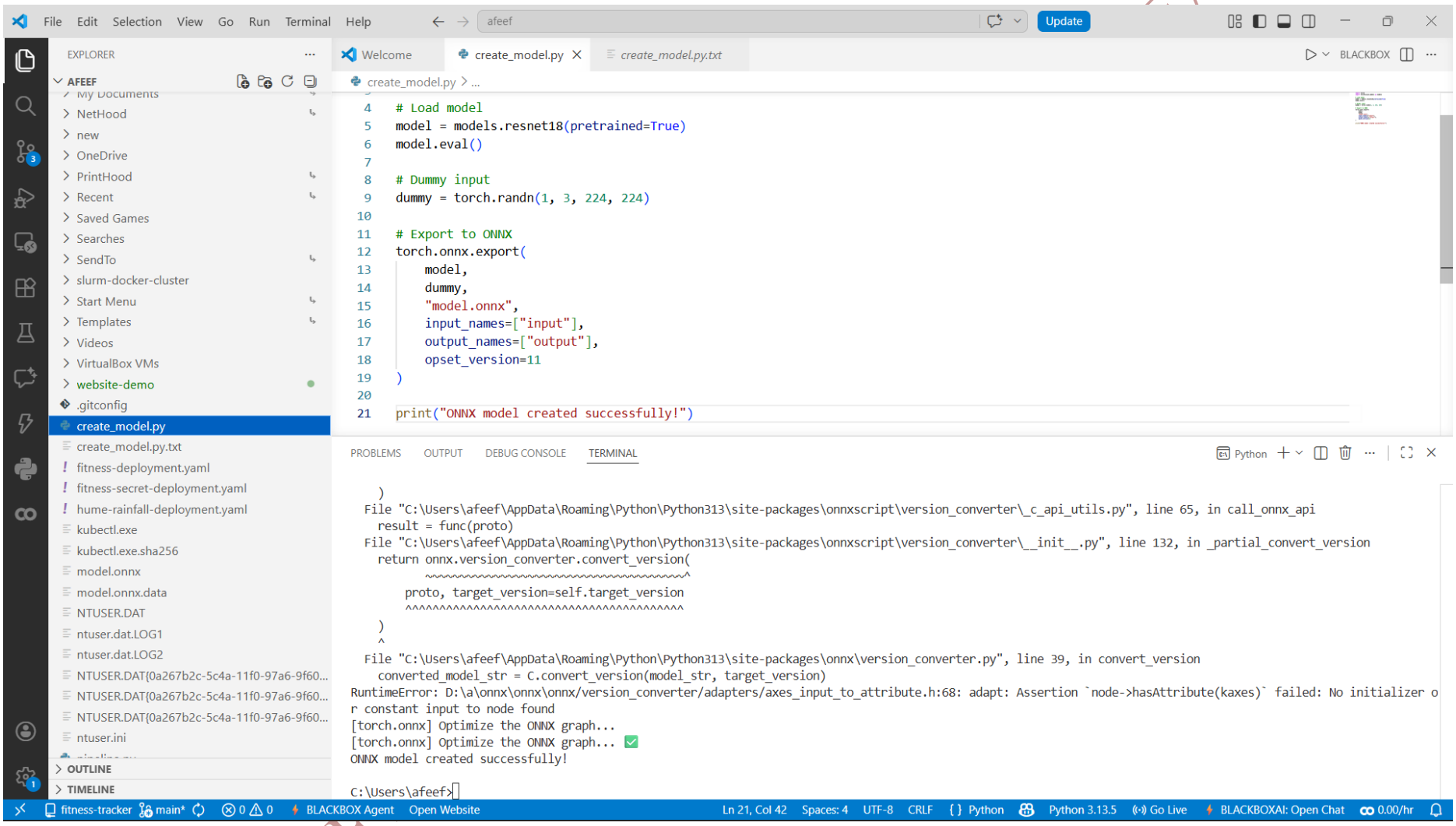

Fig 50:ONNX Model Script

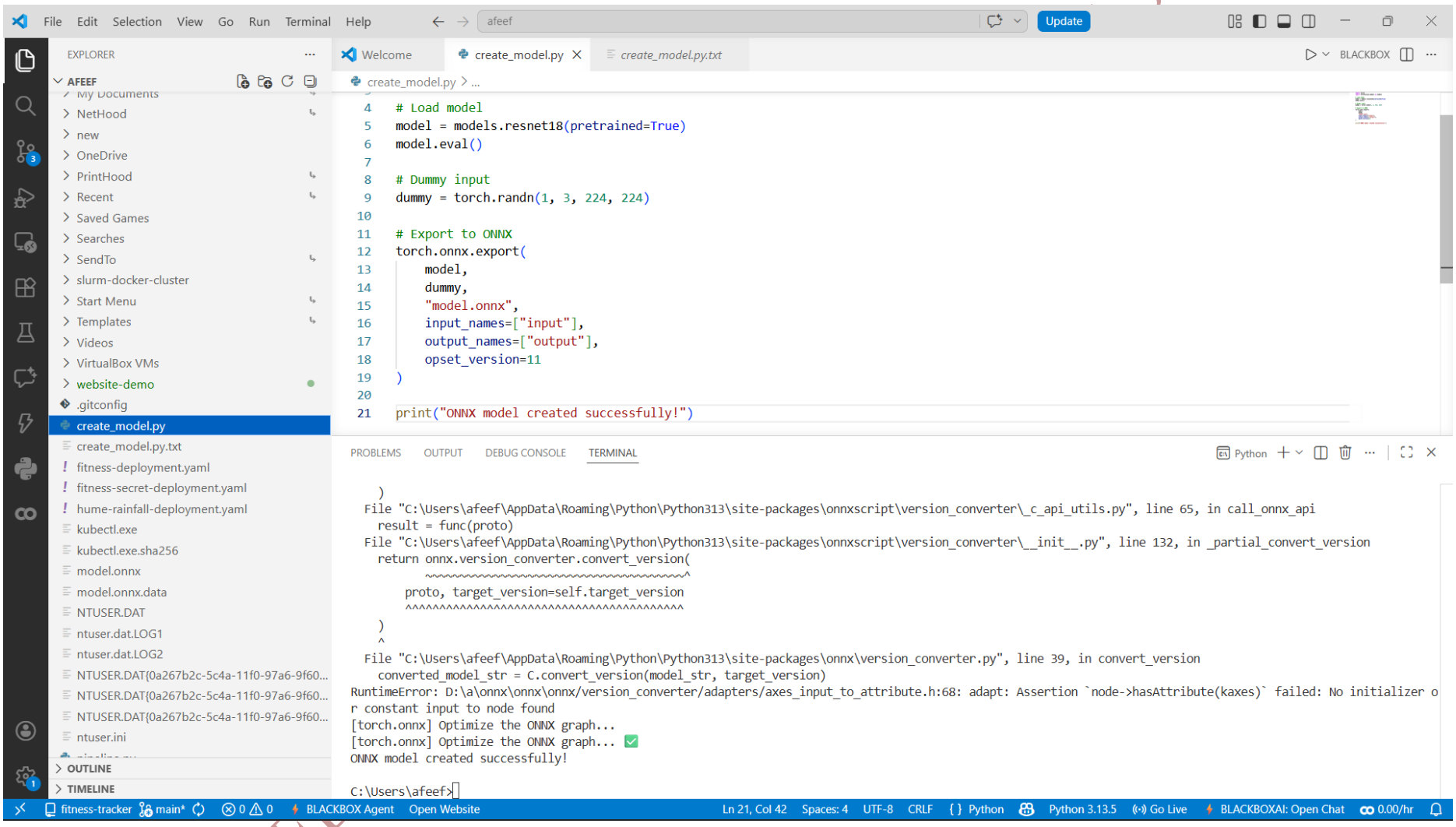

Fig 51:ONNX Model Created

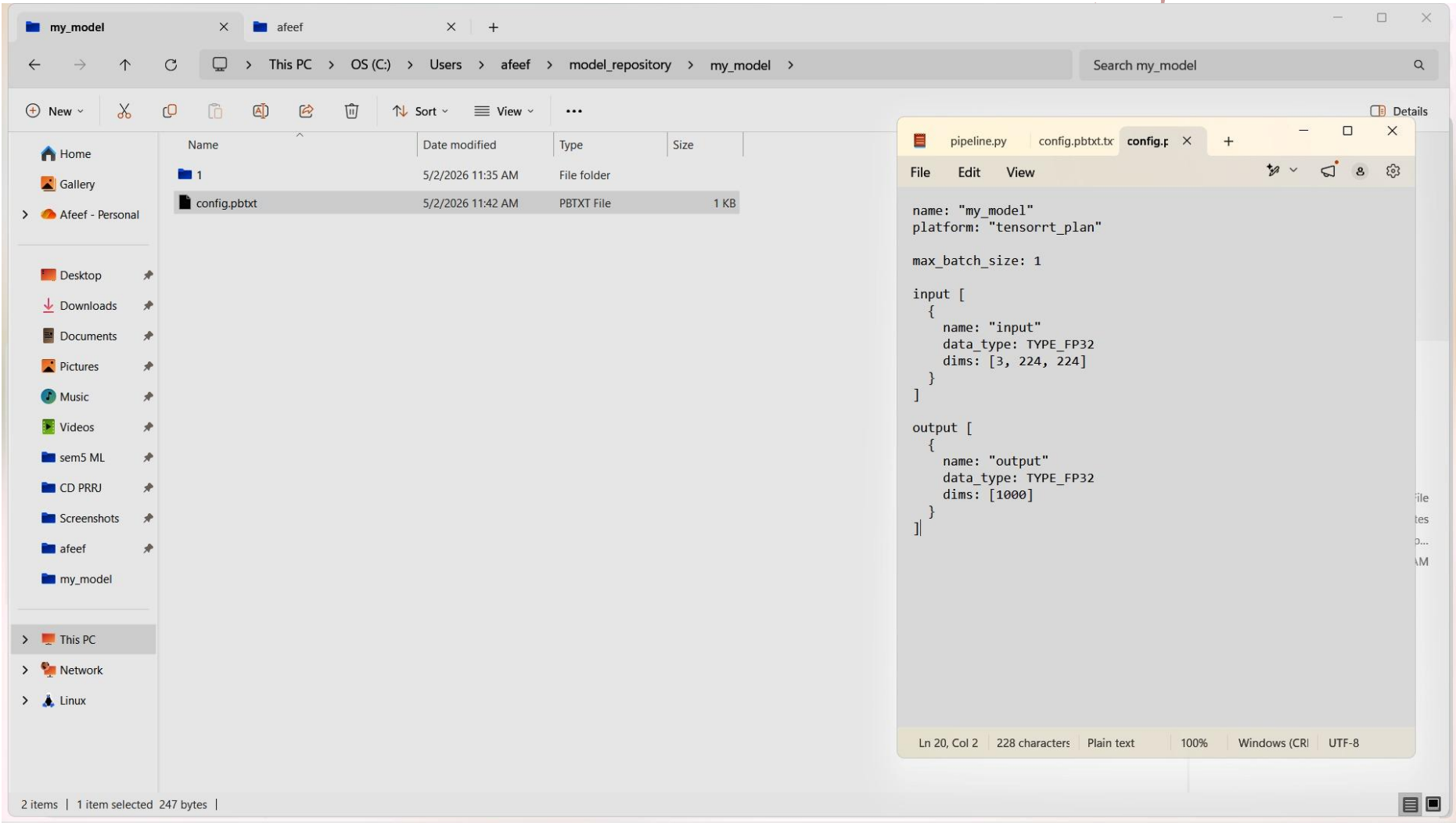

Fig 52:Triton Config File

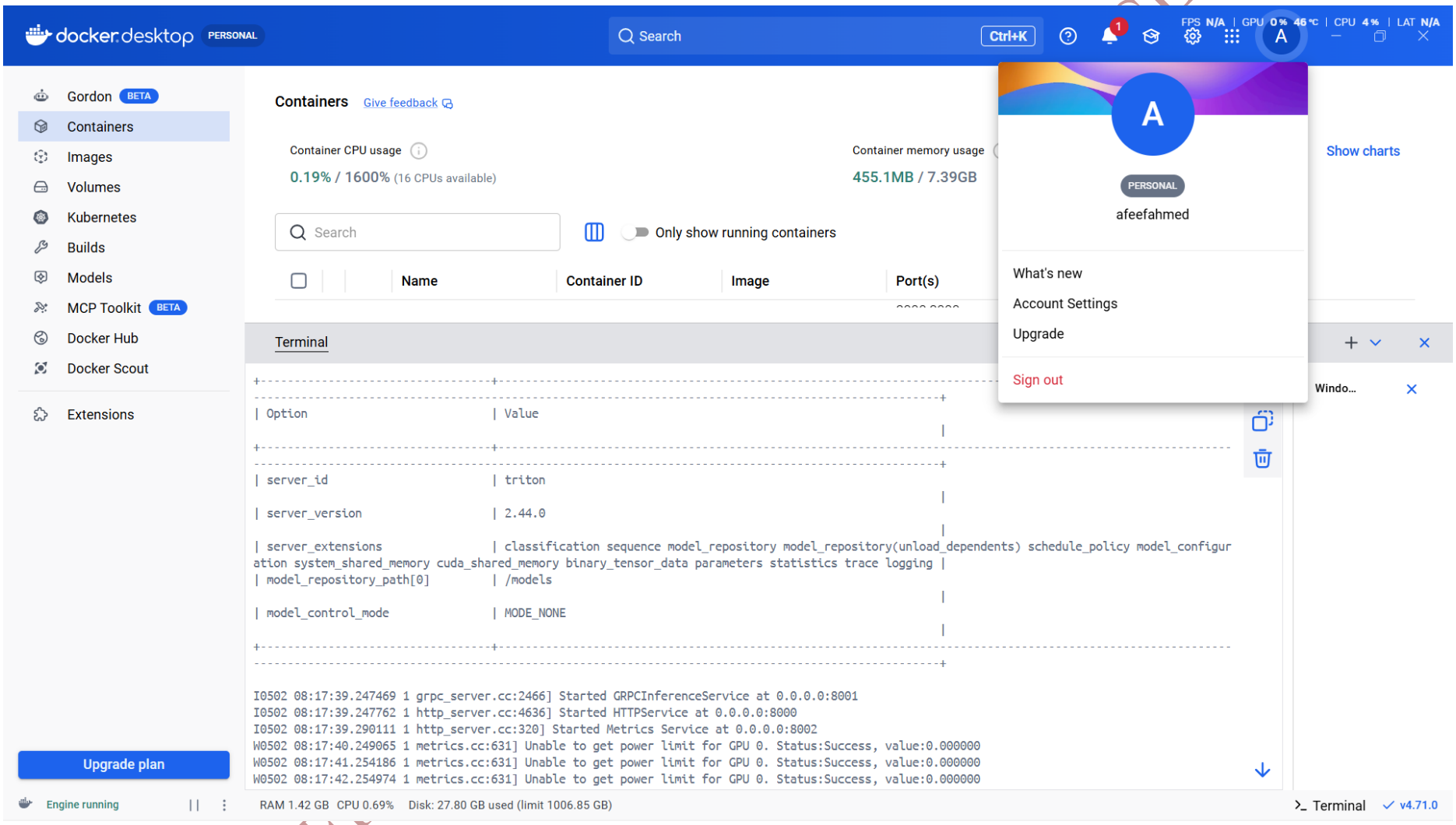

Fig 53:Triton Server Running and status

Finally, the Triton inference server was deployed using the Docker application.

```
docker run --gpus all -p8000:8000 -p8001:8001 -p8002:8002 -v C:\Users\afeef\model_repository:/models
nvcr.io/nvidia/tritonserver:24.03-py3 tritonserver --model-repository=/models
```

Server status checked.

```
curl http://localhost:8000/v2/health/ready -UseBasicParsing
```

Model is successfully deployed and Inference speed is enhanced using GPU.

Task 10 : Monitoring Load Balancers & Schedulers using Docker with Jupyter, Prometheus, and Grafana

Date: 01-5-2026

Procedure

In this experiment, load balancing and scheduling services were deployed using Docker and performance metrics were analyzed using Prometheus and Grafana visualization tools. Computational load was applied using a Jupyter notebook.

Steps Involved

In this experiment, load balancing and scheduling services were deployed using Docker Compose and visualizations were carried out using Prometheus and Grafana visualization tooling. A Jupyter notebook was used for generating computational load.

Step-by-Step Explanation

Project directory was made, and docker-compose.yml was created.

```
mkdir exp10
cd exp10
```

Docker Compose configuration consisted of following components such as Jupyter, Prometheus, Grafana and Node Exporter.

```
version: '3'
services:
  jupyter:
    image: jupyter/base-notebook
    ports: ["8888:8888"]

  prometheus:
    image: prom/prometheus
    ports: ["9090:9090"]

  grafana:
    image: grafana/grafana
    ports: ["3000:3000"]

  node-exporter:
    image: prom/node-exporter
    ports: ["9100:9100"]
```

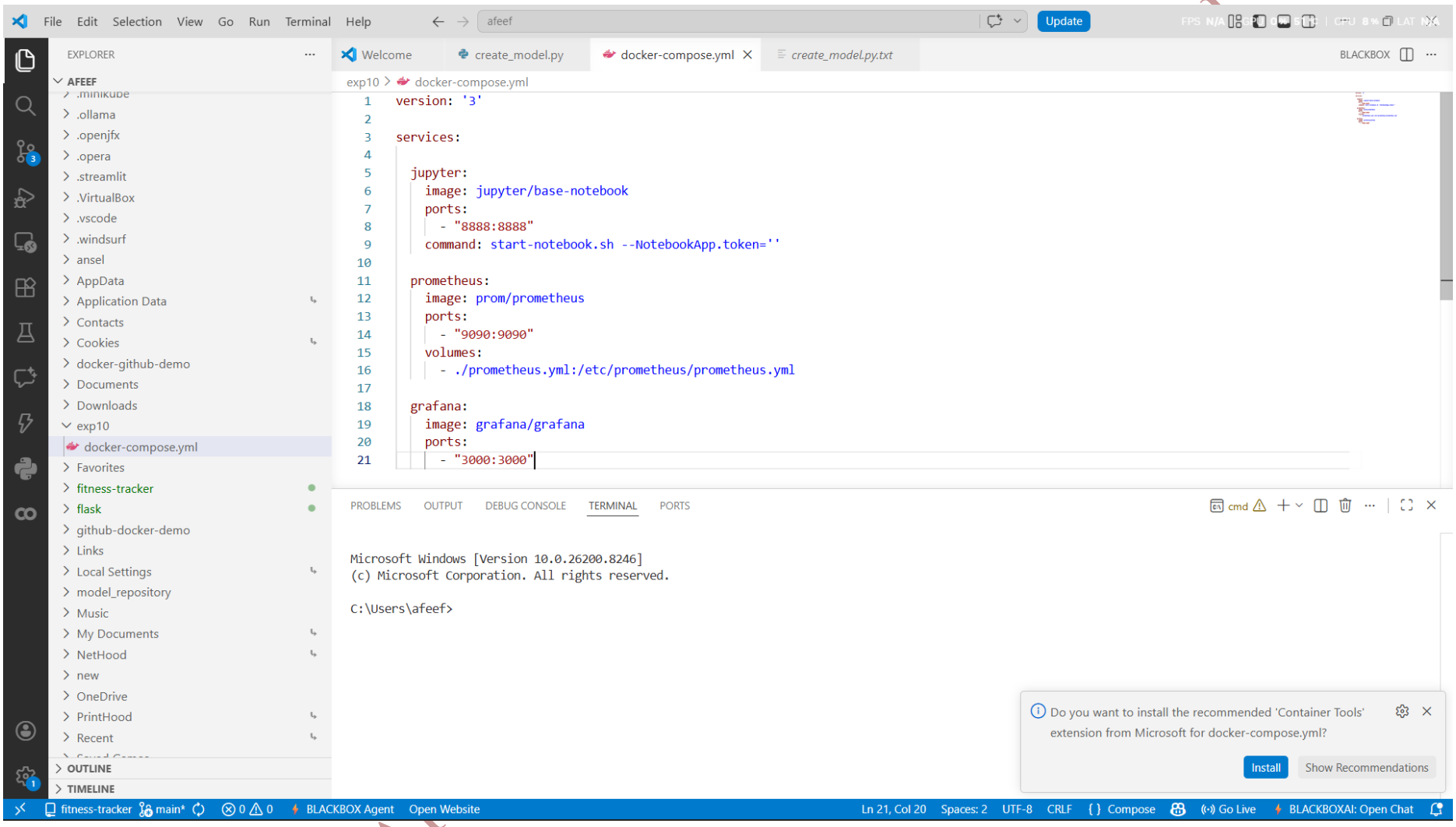

Fig 54:Docker Compose Config

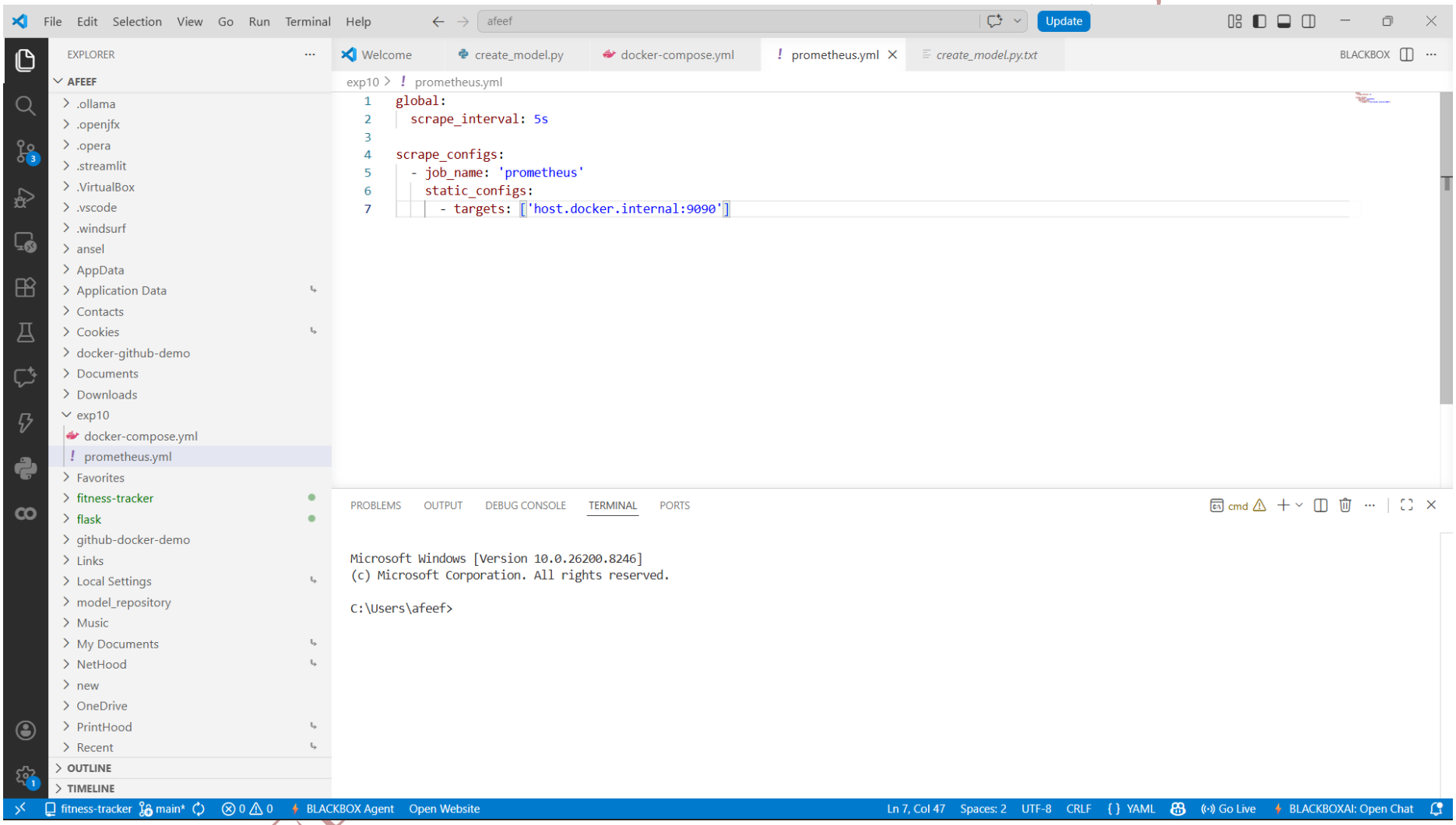

Fig 55:Prometheus Config File

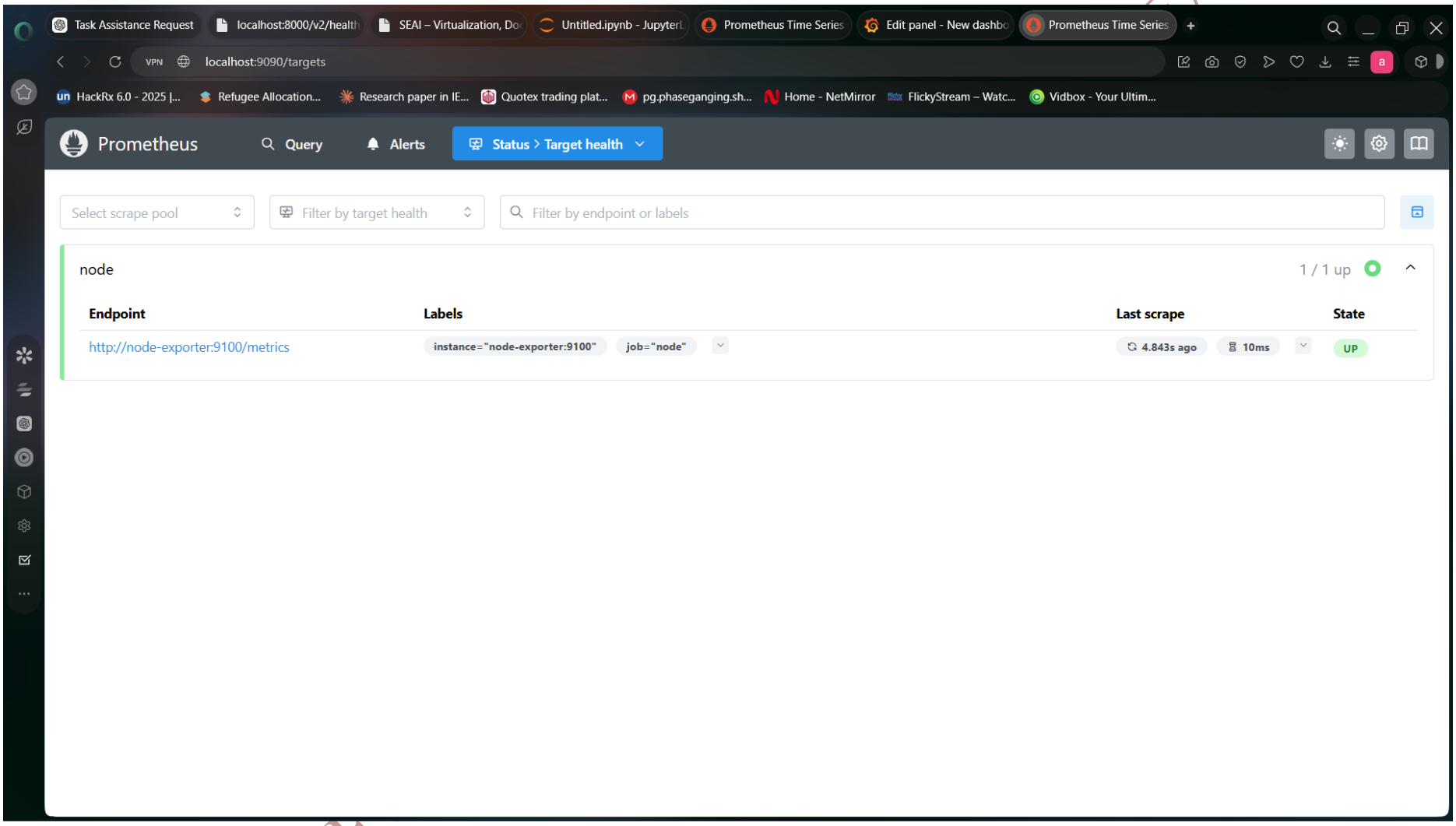

Fig 56:Prometheus Target Health Status

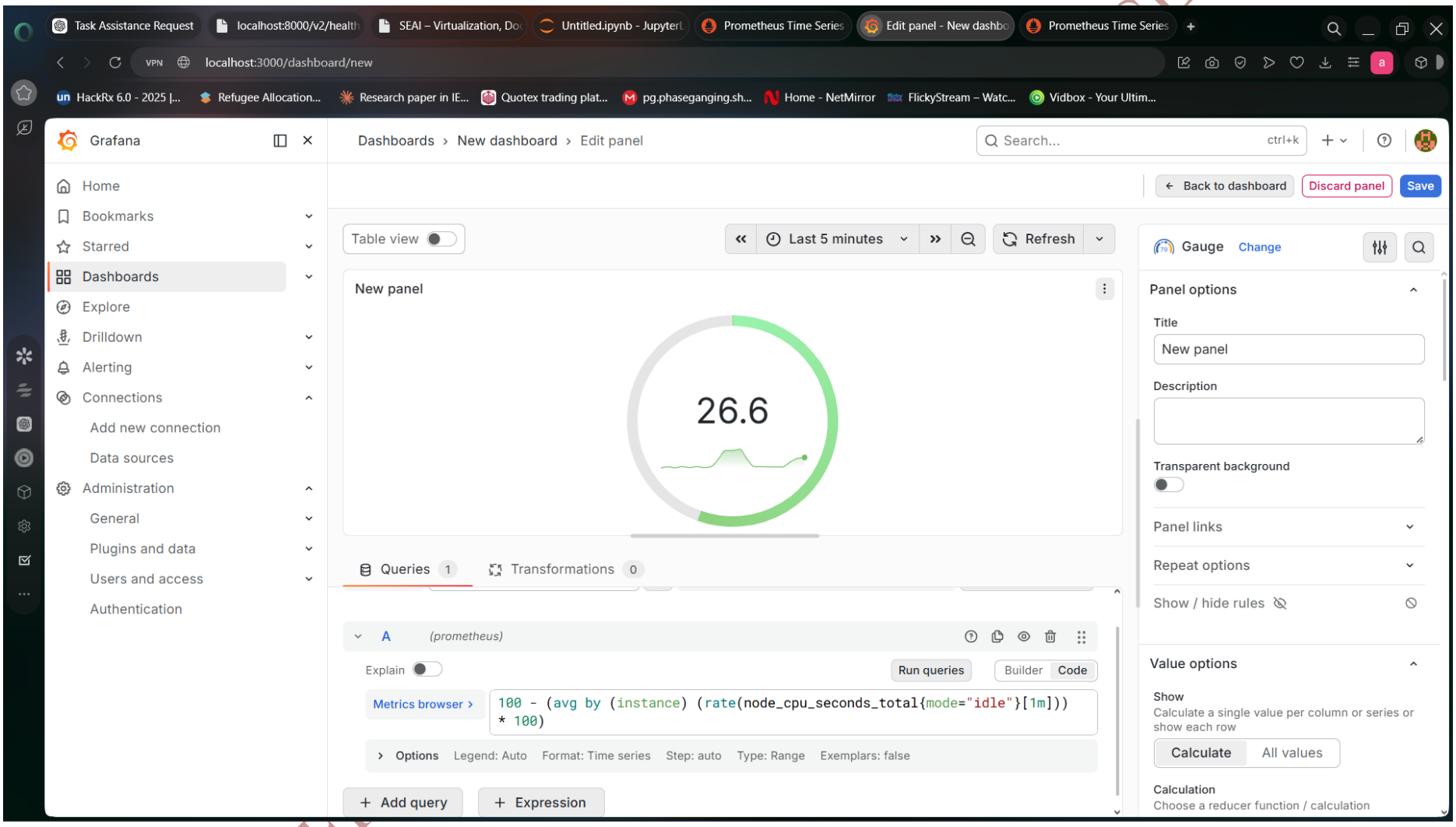

Fig 57:Grafana Monitoring Dashboard

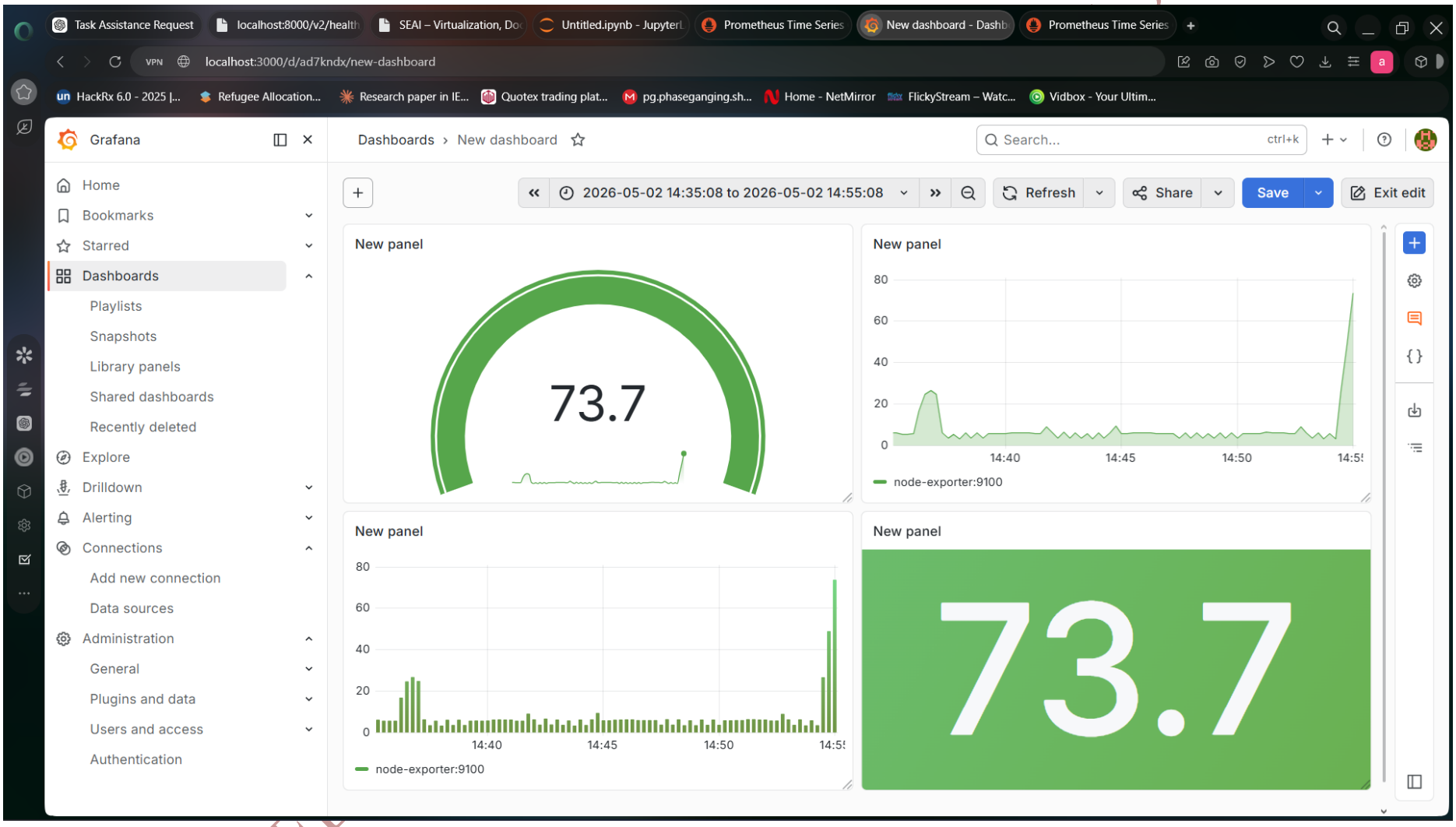

Fig 58:Grafana System Metrics Dashboard

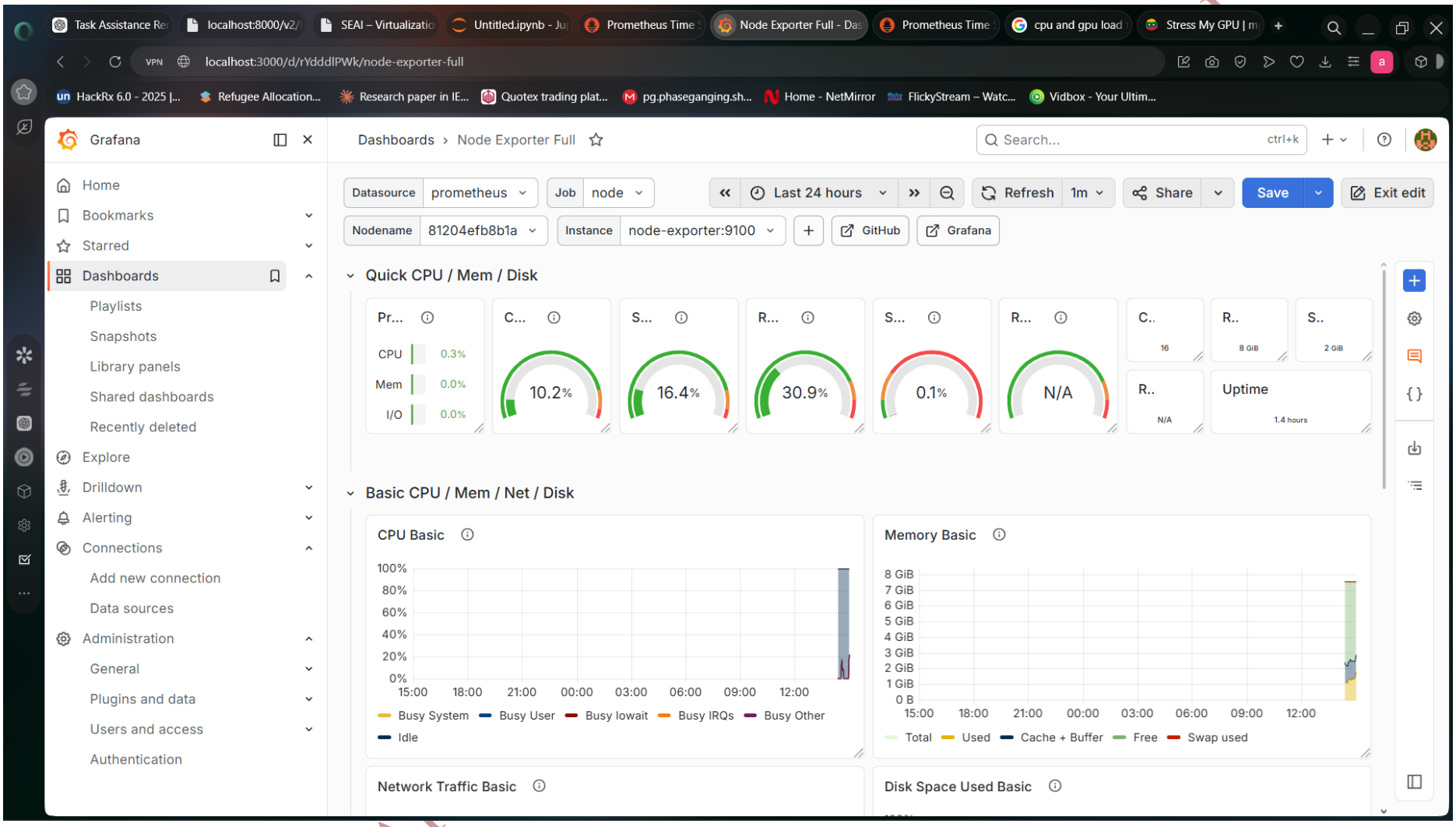

Fig 59:Grafana System Metrics Dashboard

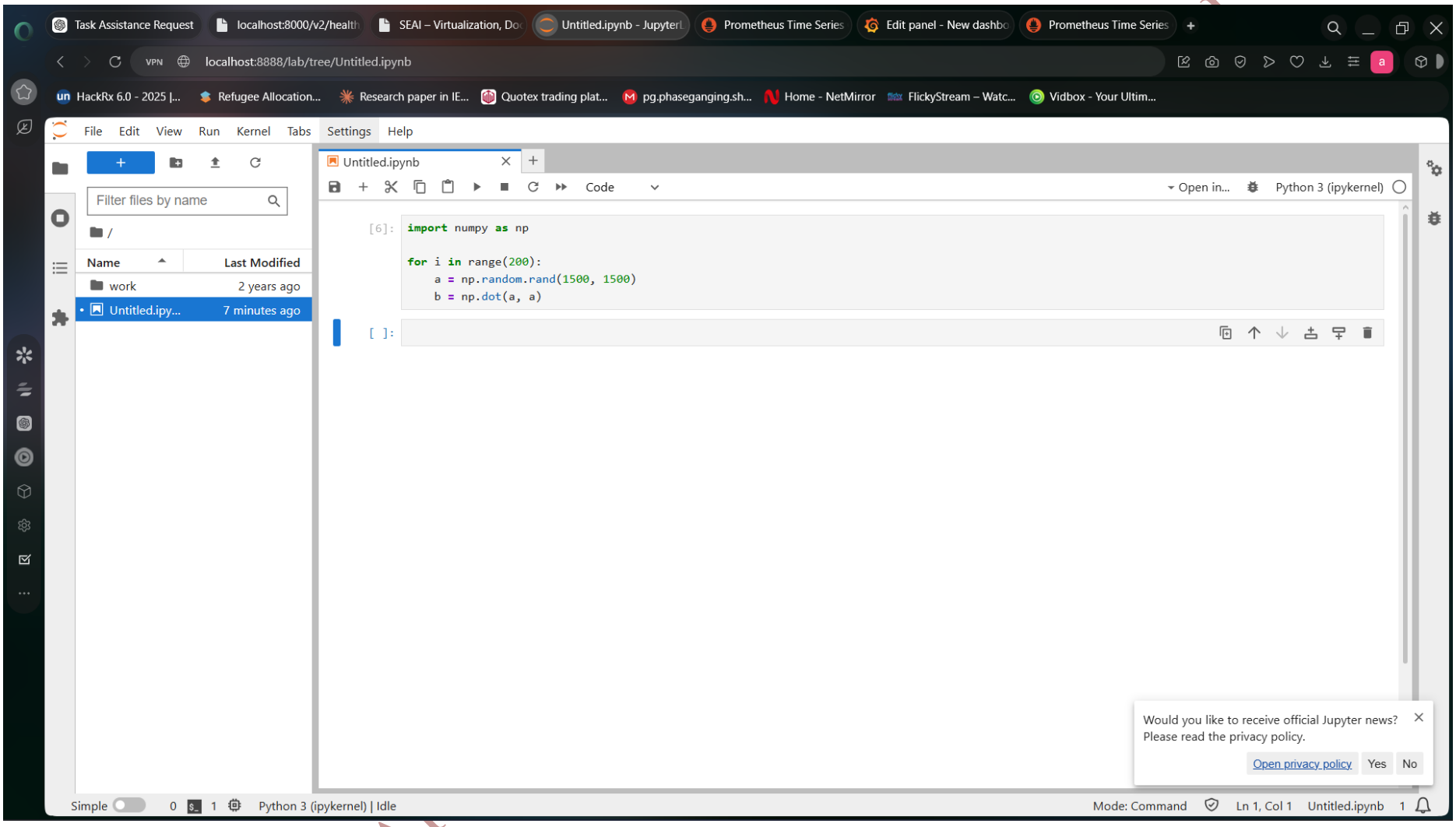

Fig 60:Jupyter Load Test Script

Services were started by using command:

```
docker compose up
```

The services were accessed through:

```
http://localhost:8888
http://localhost:9090
http://localhost:3000
```

And services can be accessed using following URLs.

```
import numpy as np

for i in range(200):
    a = np.random.rand(1500,1500)
    b = np.dot(a,a)
```

CPU usage was monitored in Grafana dashboard.